



Y3382270

学校代码: 10286
分类号: TP311
密 级: 公开
U D C: 004.4
学 号: 143383



东南大学

工程硕士学位论文

基于 FastDFS 的云存储文件系统性能 优化设计与实现

(学位论文形式: 应用研究)

研究生姓名: 朱帅

导师姓名: 罗军舟教授

朱兴明高工

申请学位类别 工程硕士 学位授予单位 东南大学

工程领域名称 软件工程 论文答辩日期 2017 年 6 月 22 日

研究方向 云存储 学位授予日期 20 年 月 日

答辩委员会主席 徐立臻 评 阅 人 吉逸 张家凤

2017 年 8 月 27 日



东南大学

硕士学位论文

基于 FastDFS 的云存储文件系统性能优化
设计与实现

专业名称: 软件工程

研究生姓名: 朱帅

导师姓名: 罗军舟 教授

导师姓名: 朱兴明 高工

PERFORMANCE OPTIMIZATION OF CLOUD STORAGE FILE SYSTEM BASED ON FASTDFS

A Thesis Submitted to

Southeast University

For the Academic Degree of Master of

Engineering

BY

Zhu Shuai

Supervised by

Prof. Luo Jun-zhou

And

Zhu Xing-ming

College of Software Engineering

Southeast University

June 2017

东南大学学位论文独创性声明

本人声明所呈交的学位论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得东南大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

研究生签名: 朱帅 日期 2017.8.31

东南大学学位论文使用授权声明

东南大学、中国科学技术信息研究所、国家图书馆有权保留本人所送交学位论文的复印件和电子文档，可以采用影印、缩印或其他复制手段保存论文。本人电子文档的内容和纸质论文的内容相一致。除在保密期内的保密论文外，允许论文被查阅和借阅，可以公布（包括以电子信息形式刊登）论文的全部内容或中、英文摘要等部分内容。论文的公布（包括以电子信息形式刊登）授权东南大学研究生院办理。

研究生签名: 朱帅 导师签名:  日期 2017.8.31

摘要

随着云计算技术的不断发展,把云存储提供的高效、安全、快捷的存储服务和柯达图文影像技术有机地融为一体是柯达未来继续立足印刷行业前沿的根本。柯达为此基于分布式文件系统 FastDFS 研发 Alpaca 云储存系统,定位于柯达全球用户通过移动客户端进行文件储存以及资源分享传递服务,实现全球柯达客户的印刷解决方案的整合。

随着移动互联网的不断普及,全球的柯达用户呈现出多通过移动互联网方式访问和使用 Alpaca 云储存系统的趋势。由于云存储的用户群体庞大且网络状况复杂,用户对于文件存储尤其是大文件存储的需求日益增强, FastDFS 分布式文件系统作为 Alpaca 云储存系统的核心底层构件,已无法满足全球 Alpaca 云存储系统用户对文件存取性能的需求。目前仍存在着诸多问题亟需解决,包括 (1) 热数据读取性能问题:大量移动端用户在访问热数据时,系统会出现资源消耗过大、高延迟以及数据传输速度慢等问题。(2) 服务器集群负载均衡问题:在云存储全球化的趋势下,存储系统对负载均衡算法的性能提出了更高的要求,传统的静态负载方式已无法满足云储存系统全球化应用场景的需求。(3) 大文件存储问题:大文件在上传过程中将耗费存储系统大量的上传时间以及流量,同时大文件的冗余备份会占用较多的 CPU 时间,影响存储节点服务性能。

为了解决上述问题,本文研究并设计了基于 FastDFS 性能优化的 Alpaca 云存储系统,主要工作包括以下三个方面 (1) 针对 FastDFS 热文件读取性能低的问题,本文研究和制定包括缓存区开辟前置条件、文件查找流程、缓存区文件淘汰机制、缓存区更新策略等一系列的缓冲区管理规则,设计并实现了缓冲机制管理和优化模块。(2) FastDFS 分布式文件系统只有轮询、选择最大剩余空间、指定服务器等几种简单的静态负载均衡方式,无法满足 Alpaca 云存储系统全球化应用场景的需求,因而,针对服务器集群负载均衡问题,本文结合了 CPU、内存、磁盘、网络带宽以及负载均衡等相关因素,应用现有的动态负载均衡算法研究成果,设计并实现了负载均衡管理和优化模块。(3) FastDFS 分布式文件系统出于设计精简的考虑,未对文件做分块存储及数据块去重处理,因而,针对大文件存储问题,本文通过应用现有的文件分块以及数据去重算法研究成果,形成新的 FastDFS 文件存取机制,实现了大文件存储管理和优化模块。

基于 FastDFS 性能优化的 Alpaca 云存储系统使用 C 语言在 Linux 环境下进行实现,在 Alpaca 云存储系统中使用了缓存技术、负载均衡技术以及大文件重复数据删除与分块技术,实现了缓冲机制管理和优化、负载均衡管理和优化以及大文件存储管理和优化三大模块并与 Alpaca 云存储系统进行集成,达到了高效处理热数据、均衡服务器集群负载请求以及提高大文件存储效率的设计目标。

关键词: 分布式文件系统, 重复数据删除技术, 缓存技术; 负载均衡技术

Abstract

With the development of cloud computing technology, Kodak's future foothold in the forefront of the printing industry is that efficient, safe, fast storage services provided by cloud storage and Kodak image technology are organically integrated into one, Kodak has developed Alpaca cloud storage system based on distributed file system FastDFS, which aims to enable Kodak global users to use file storage and resource sharing service through mobile client and realize the integration of Kodak customers' printing solutions

With the growing popularity of mobile Internet, Kodak users around the world have a tendency to access and use Alpaca cloud storage system through mobile internet. As cloud storage user groups are large and the network is complex, there is a growing demand for it. FastDFS distributed file system is the core underlying component of Alpaca cloud storage system, which was unable to meet the needs of global users for the performance of Alpaca cloud storage system. Since FastDFS is designed for traditional Internet, there are still a number of problems that need to be addressed when it is applied to mobile Internet. (1) Hot data reading performance. When a large number of mobile client users access those hot data, the system will appear too much resource consumption, high latency and slow data transmission and so on. (2) Load balance performance for server cluster. In the trend of cloud storage globalization, the storage system has put forward higher requirements for the performance of load balance algorithm, and traditional load balance algorithm can not meet the requirements of globalization application scenario of cloud storage system. (3) Large file storage performance. First of all, large file will cost cloud storage system a lot of time and traffic in the upload process. Then, large file backup will spend more CPU time. At last, these issues affect the performance of storage node.

In order to solve above issues, the paper studies and designs Alpaca cloud storage system based on FastDFS, the main tasks are as follows. (1) For Low reading performance of FastDFS hot file, some cache management rules have been studied and formulated in the paper, such as precondition of cache is used, file search process, cache elimination mechanism and cache update strategy. At last, a cache management and optimization module is designed and implemented in the paper. (2) FastDFS only has several simple static load balance algorithm, such as polling, selecting the largest residual space and designating server, which can not meet the needs of Alpaca cloud storage system. In the paper, some important load balance factors, such as CPU, memory, disk, network bandwidth, are applied to the existing dynamic load balance algorithm for the sake of load balance performance. At last, the paper designs and implements a load balance management and optimization module. (3) FastDFS does not consider data chunking technology and data deduplication technology for the sake of simplification. Accordingly, the paper introduces a new file access mechanism by applying the existing data chunking algorithm and data deduplication algorithm, and implements a large file storage management and optimization module.

Alpaca cloud storage system based on FastDFS is implemented in Linux environment, and cache technology, load balance technology, data deduplication technology are used. Three modules, such as cache management and

optimization module, load balance management module and large file storage management and optimization module, are implemented in the paper and integrated with Alpaca cloud storage system, it achieves the design goal of dealing with hot data, balancing server cluster traffic, and improving the efficiency of large file storage

Keywords distributed file system, deduplication, cache, load balance

目录

摘要	I
Abstract	II
第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 云存储现状	2
1.2.2 分布式文件系统现状与分析	2
1.3 研究内容	5
1.3.1 基于 FastDFS 的云存储文件系统优化机制设计	5
1.3.2 基于 FastDFS 的云存储文件系统架构及功能模块设计与实现	6
1.3.3 系统验证与实现	6
1.4 论文组织结构	6
第二章 相关技术基础	8
2.1 FastDFS 体系结构	8
2.1.1 FastDFS 中三个基本角色	9
2.1.2 FastDFS 三大特色	9
2.1.3 小文件存储及文件命名策略	10
2.1.4 负载均衡方式	11
2.1.5 文件上传	11
2.1.6 文件下载	11
2.2 关键技术分析	12
2.2.1 缓存技术	12
2.2.2 负载均衡技术	13
2.2.3 重复数据删除及大文件分块技术	15
2.3 本章小结	17
第三章 系统需求分析	18
3.1 系统总体功能需求分析	18
3.2 业务场景功能需求分析	19
3.2.1 缓存机制管理和优化模块	19
3.2.2 负载均衡管理和优化模块	19
3.2.3 大文件存储管理和优化模块	20
3.3 本章小结	21
第四章 Alpaca 云存储系统模块设计与实现	22
4.1 开发环境	22
4.2 Alpaca 云存储系统总体框架	22
4.3 缓存机制管理和优化模块	23
4.3.1 缓存管理算法	23
4.3.2 缓存区管理机制	24
4.3.3 缓存区下载文件流程描述	25
4.3.4 缓存区文件更新与删除	26
4.3.5 缓存区数据结构	27
4.4 负载均衡管理和优化模块	27

4.4.1 动态反馈机制28

4.4.2 新负载均衡算法具体应用29

4.4.3 新负载均衡算法伪代码描述29

4.4.4 负载均衡算法数据结构30

4.5 大文件存储管理和优化模块30

4.5.1 重复数据删除技术实现方式31

4.5.2 Fingerdiff 算法具体实现31

4.5.3 文件分块实现31

4.5.4 文件上传32

4.5.5 文件下载32

4.5.6 文件删除33

4.5.7 重复数据删除及大文件分块数据结构.....33

4.6 本章小结35

第五章 Alpaca 云存储系统模块测试36

5.1 测试环境36

5.1.1 测试环境架构36

5.1.2 改进的 FastDFS 环境部署37

5.2 测试方案39

5.2.1 模块性能测试39

5.2.2 系统性能测试42

5.3 本章小结44

第六章 总结与展望45

6.1 总结45

6.2 展望45

参考文献47

致谢50

实习情况介绍51

第一章 绪论

本章介绍了项目研究的背景与意义，简述了云存储以及分布式文件系统的发展历史、技术特征以及研究现状，确定了项目的研究内容以及对项目的设计与实现做了简要的描述，最后对论文的结构以及主要工作作了安排。

1.1 研究背景与意义

柯达在材料科学、沉积科学以及影像科学有着强大的技术遗产，无论是为优异的功能性油墨开发新的材料，还是为了将油墨准确高速地转移到承印物上而设计新的印刷引擎，柯达研发中心始终走在科技和应用的最前沿。然而，这些技术要创造出强大的生产力，就必须依托高效的软件解决方案来管理生产流程。在云计算如火如荼的今天，把云存储技术提供的高效、安全、快捷的服务和柯达图文影像技术有机地融为一体是柯达未来继续立足行业前沿的根本。

为此，柯达集中其产品研发部门的各个项目小组，研发柯达 Alpaca 云储存系统。柯达 Alpaca 云储存系统主要目的是用于整合全球柯达客户的印刷解决方案，实现对海量印刷过程中所涉的设备色彩特性文件、DeviceLink 文件，商业印刷中的包装、封面、LOGO 以及版式等设计稿件，个性化影像业务和景点影像解决方案形成的影像文档等各类文件的高效存储管理，并通过 Alpaca 云储存客户端便捷地分享给全球的柯达用户，提升柯达的服务品质。本选题依据正是 Alpaca 云储存系统的实际项目需求以及市场调研的结果。

Alpaca 云储存系统定位于柯达全球用户通过移动客户端进行文件储存以及资源分享传递服务。根据统计，Alpaca 云储存系统所存储的文件中，70%的文件小于 300M，20%的文件在 300M-1G 之间，10%的文件大于 1G，其客户端由 Android 客户端、iOS 客户端、Windows phone 客户端以及 web 客户端组成。分布式文件系统是用来给云存储提供底层支撑以及实现各种文件管理功能的接口，鉴于 Alpaca 云储存系统的应用场景以及客户需求，在分布式文件系统的选择上，分布式文件系统的可扩展性、文件读取效率以及复杂网络环境下系统的稳定性以及可靠性等方面显得尤为重要。FastDFS^[1]作为一个开源的轻量级分布式文件系统，具备大容量存储、高并发访问、硬件成本较低、代码开源等特点，适合用于存储和管理柯达用户在生产过程中所使用和产生的海量图片、文档等文件，并提供文件为载体的在线服务。因此，柯达使用 FastDFS 做为 Alpaca 云储存系统中的分布式文件系统。

然而，随着移动互联网的不断普及，全球的柯达用户呈现出多通过移动互联网的方式访问和使用 Alpaca 云储存系统的趋势。众所周知，在移动数据都存储在云端时代，文件的远程网络访问已成为一种常态，云存储用户通常群体庞大且网络状况复杂，用户对于文件存储尤其是大文件存储的需求日益增大，FastDFS 分布式文件系统已无法满足全球 Alpaca 云储存系统用户对文件存取性能的需求。首先，在移动互联网环境中，文件的分享与读取相对传统互联网要频繁许多。对于被高频次访问的文件，FastDFS 分布式文件系统每次均从磁盘读取，加重系统负载的同时影响用户体验。其次，FastDFS 分布式文件系统只有轮询、选择最大剩余空间、指定服务器等几种简单的负载均衡方式，无法满足 Alpaca 云储存系统的全球化应用场景对负载均衡的需求。另外，FastDFS 分布式文件系统出于设计精简的考虑，未对文件做分块存储及数据块去重处理，在文件上传过程中将耗费大量不必要的上传时间以及流量，同时 FastDFS 大文件的冗余备份会占用较多的 CPU 时间，影响存储节点服务性能。不难看出，

FastDFS 分布式文件系统在文件缓存、请求负载均衡、大文件存储等方面的实现存在一定设计缺陷,对 Alpaca 云储存的整体文件存取性能造成关键影响。为此,如何使 FastDFS 满足 Alpaca 云存储系统用户在文件缓存、请求负载均衡、大文件存储等方面的性能需求,提高被用户频繁访问文件的访问效率,保证全球 Alpaca 云储存用户的大量文件请求得到高效响应,保障 Alpaca 云储存用户大文件上传下载的稳定性及可靠性,成为 FastDFS 分布式文件系统成功应用于 Alpaca 云储存系统所亟待解决的问题。

1.2 国内外研究现状

随着全球信息化程度不断提升,数据量呈现爆炸式增长。面对互联网的海量数据存储需求,云存储以其性价比高、扩展性强、可用性好等优势获得行业的认同。云存储通常采用可扩展的分布式文件系统作为底层数据存储构件,并使用价格低廉的服务器进行系统部署,通过添加服务器即可实现存储能力的线性扩充,从而使得整体存储性能灵活可扩展的同时保持较低的成本。

1.2.1 云存储现状

随着 Internet 技术的发展,数据量呈现爆炸性增长,数据存储逐渐成为制约企业发展的关键问题,因而越来越多的企业开始将数据存储分离出来作为独立的项目进行管理,从而出现了云存储技术。云存储的兴起使得整个 IT 界处于一个重大的变革期,从以应用程序、设备为中心转向以数据为中心,而且这一转变还将影响到技术和业务两个层面。云存储的最大特点是海量数据存储能力、高可用性以及低成本,而其带来的最大变革是,云存储提供商以一定粒度拆分云存储的数据存储能力,逐渐从销售软件解决方案转向根据软件解决方案的实际使用收费,也就是从卖产品转向了卖服务。因此,可以说云存储不是存储而是服务。

随着云存储技术的不断发展,很多 IT 界巨头纷纷推出基于云存储的不同服务。例如,在谷歌的云应用平台,谷歌用户可以使用 Google Docs 服务对文档进行在线编辑,并将其保存至谷歌提供的云存储空间。无论身在何处,谷歌用户都可以通过云存储系统实现文档的共享、传送、以及版权管理等功能。除此之外,谷歌还推出 Google Music 服务,允许用户保存上万首音乐。另外,Apple 公司也推出了 iCloud 云服务,它是一个整合型云服务产品,包含了云计算,云搜索,云存储等多种整合功能,用户可以通过苹果设备使用 iCloud 服务存取照片、应用、日历、文章、邮件、电子书、音乐等数据。iCloud 以全球超过 2 亿的用户作为后盾,将所有 IOS 终端集合起来,使其成为一个新型的社交网络。如 Google、Apple 云储存服务给用户带来数据存储与管理的便捷性以及安全性,是传统存储方式所无法比拟的。

纵观国外云存储的应用发展趋势以及借鉴各大互联网存储服务商的成功案例,国内云存储行业也正加速升温,各大互联网公司纷纷推出各种云存储服务。国内基于手机移动数据存储以及网盘数据存储的云存储服务发展蓬勃,腾讯微云、小米云盘、百度云盘等云存储应用大幅扩张业务。目前国内的互联网公司大多都以产品附加自家云存储服务的模式去抢夺亿级用户市场,未来云存储的发展趋势也越来越广,国内竞争将越来越激烈。

1.2.2 分布式文件系统现状与分析

云存储的兴起正在颠覆现有的网络存储架构,面对当前海量的数据存储需求,传统的 SAN 或 NAS 在性能和

容量的升级上会存在瓶颈。现有的网络存储架构一旦遭遇到瓶颈，就会不断地促使企业升级到性能更好的存储系统和增加更多的存储服务器以及管理人员，由此增加了成本。

今天，互联网企业每天都需要存储海量数据，频繁的数据传送对 I/O 性能要求非常高，传统的存储架构对此已无能为力。近几年兴起的分布式存储技术，不仅突破了传统 SAN 架构的性能瓶颈，而且可以实现性能与容量的线性扩展，这对于追求高可用性、高性能以及高经济效益的企业用户来说是一个新选择。从前沿的互联网公司的实践来看，它们在现有的云计算环境中并没有采用 SAN 架构，而是使用了可扩展的分布式文件系统[2]。

在云计算和云存储的需求拉动下，可扩展的分布式文件系统呈现出百家争鸣的趋势，越来越多的优秀的分布式文件系统出现在大众面前，它们虽在特定的应用下能胜任当前的任务，但也各有优缺点。目前比较常见的分布式文件系统有 HDFS^[2]、GFS^[3]、Lustre^[4]、GridFS^[5]、MogileFS^[6]、TFS^[7]、NFS^[8]、OpenAFS^[9]、FastDFS 等。

(1) HDFS

Hadoop 分布式文件系统(HDFS)被设计成适合运行在通用硬件上的分布式文件系统。HDFS 相比其他文件系统有三个比较显著的优点，首先，它可以存储很大的文件，典型的 HDFS 文件大小是 GB 到 TB 的级别；另外，HDFS 被设计成适合进行文件批量处理的系统，而不是用户交互式的，最后，故障的检测和自动快速恢复是 HDFS 一个很核心的设计目标，所以在 HDFS 中，硬件故障被当做常态，而不是异常。

HDFS 是一个性能优越的分布式文件系统，但是 HDFS 的缺点也相对明显。首先，HDFS 设计用于有大吞吐量数据的场景，这是以一定延时为代价，因而无法满足对响应速度要求高的移动互联网应用程序的需求；另外，移动互联网应用程序在被使用过程中会产生海量非结构化的小文件，而且文件在线修改已是常规性需求，HDFS 无法高效存储海量小文件以及不支持多用户写入及任意修改文件，这决定了 HDFS 不适合移动互联网的应用场景。

(2) GFS

GFS 是 Google 公司开发的一个可扩展的分布式文件系统，用于大型的、分布式的、对大量数据进行访问的应用。它运行于廉价的普通硬件上，并提供容错功能。它可以给大量的用户提供总体性能较高的服务。GFS 与以往的文件系统有一些不同的观点，例如，部件错误不再被当作异常，而是将其作为常见的情况加以处理，对于文件管理，设计中对大型的文件的管理一定要能做到高效，对小型的文件也必须支持，但不必优化，对于文件的更新，GFS 是通过添加新数据完成的，而不是改变已存在的数据，另外，GFS 设计者认为，高可持续带宽比低延迟更重要。

GoogleFS 是只有一个主服务器的文件系统，这就要求主服务器是所有节点中性能最好的节点。整个系统的最大存储容量和正常工作时间受制于主服务器的容量和正常工作时间，与文件操作有关的所有动作和请求都经过主服务器，这将极大地限制系统可扩展性和可靠性。

(3) Lustre

Lustre 是 HP, Intel, Cluster File System 公司联合美国能源部开发的 Linux 集群并行文件系统。它是第一个基于对象存储设备的开源并行文件系统。它由客户端，两个 MDS，OSD 设备池通过高速的以太网或 QWS Net 所构成。可以支持 1000 个客户端节点的 I/O 请求，两个 MDS 采用共享存储设备的 Active—Standby 方式的容错机制，Lustre 存储设备跟普通的、基于块的 IDE 存储设备不同，是基于对象的智能存储设备。

Lustre 只能部署在 Linux 特定内核的平台上，需要重新编译内核，需要特殊设备的支持，这将极大限制 Lustre

的普及以及云存储系统的扩展性。而且, Lustre 分布式的元数据服务器管理还没有实现, 比较适应于大文件的储存, 无法满足移动互联网应用的海量非结构化小文件的存储需求。

(4) MogileFS

MogileFS 是一套高效的文件自动备份组件, 由 Six Apart 开发, 广泛应用在包括 LiveJournal 等 web2.0 站点上。主要特征包括 无需特殊的核心组件、无单点、自动文件复制、传输中立, 无特殊协议、简单的命名空间、不共享任何数据等。

FastDFS 设计时借鉴了 MogileFS 的一些思路, 相对于 MogileFS, FastDFS 具有如下特点和优势 和 MogileFS 相比, FastDFS 裁减了 storage server 调度用的数据库, FastDFS 只有 tracker server 和 storage server 两个角色, 这既简化了系统, 同时也消除了数据库带来的性能瓶颈, 另外, FastDFS 和 MogileFS 相比, 没有文件索引数据库, FastDFS 整体性能更高; 最后, 从采用的开发语言上看, FastDFS 用 C 语言编写, MogileFS 用 perl 编写, FastDFS 比 MogileFS 更底层、更高效。

(5) GridFS

GridFS 是一种将大型文件存储在 MongoDB 的文件规范, 数据库支持以 BSON 格式保存二进制对象。但是 MongoDB 中 BSON 对象最大不能超过 16MB。GridFS 规范提供了一种透明的机制, 可以将一个大文件分割成为多个较小的文档。这将容许我们有效的保存大的文件对象。它把文件分割成小块, 大小通常在 256K 的大对象。

GridFS 是基于 MongoDB 的文件存储系统, GridFS 用分块存储的形式存储文件, 每块大小为 255KB。数据存放在两个表里, 一个表是 chunks, 加上元信息后单条记录在 256KB 以内, 另一个表是 files, 存储文件元信息。通过研究发现, GridFS 有两个比较明显的缺点, 第一, GridFS 访问文件是从 MongoDB 中读取, 这种方式要比直接从磁盘上读取文件要慢。另外, GridFS 修改文件, 只能先将已有文件删除, 然后再将整个文件重新保存。GridFS 文件存取方式只适应于具有较低负载压力的应用场景, 在高负载的情况下, GridFS 将会极大地限制移动互联网云存储系统对文件实时存取的需求。

(6) TFS

TFS (Taobao FileSystem) 是阿里巴巴集团研发的一款可扩展、高性能的分布式文件系统, 其主要设计目标是海量小文件存储, 并可为外部应用提供高并发和高可用的文件存储服务。它采用了 HA 架构和平滑扩容, 保证了整个文件系统的可用性和扩展性。同时扁平化的数据组织结构, 可将文件名映射到文件的物理地址, 简化了文件的访问流程, 一定程度上为 TFS 提供了良好的读写性能。

(7) NFS

NFS 即网络文件系统, 它允许网络中的计算机之间通过 TCP/IP 网络共享资源。本地 NFS 客户端应用可以透明地读写位于远端 NFS 服务器上的文件, 就像访问本地文件一样。它的优点是: 只要有一台能够连接网络的机器上存储了某个目标文件, 用户就可以通过 NFS 访问这台机器上的这个文件, 因而用户不必在每台机器上都存储冗余文件, 达到释放本地磁盘存储空间的目的, 另外, 一些硬件设备可以在网络上面被别的机器使用, 减少了整个网络上的可移动介质设备的数量。

NFS 允许在多个用户之间共享公共文件系统, 并提供数据集集中的优势, 来最小化所需的存储空间, 但他并不是适合对数据安全敏感性要求较高的个人云存储需求。另外, 在高并发的环境 NFS 效率性能有限, NFS 客户端访

问认证的时候基于 IP 和主机名，安全性较差；NFS 容易存在单点故障。这些都严重制约着 NFS 在云存储领域的应用。

（8）OpenAFS

OpenAFS 是一套开放源代码的分布式文件系统，允许系统之间通过局域和广域网来分享档案和资源。OpenAFS 是围绕一组叫做 cell 的文件服务器组织的。每个服务器的标识通常是隐藏在文件系统中的。从 AFS 客户机登录的用户将分辨不出他们在哪个服务器上运行，因为从用户的观点来看，他们想在有可识别的 UNIX 文件系统语义的单个系统上运行。文件系统内容通常都是跨 cell 复制，以便一个硬盘的失效不会损害 OpenAFS 客户机上的运行。OpenAFS 需要高达 1 GB 的大容量客户机缓存，以允许访问经常使用的文件。它还是一个十分安全的基于 Kerberos 的系统，它使用访问控制列表 (ACL) 以便可以进行细粒度的访问。

OpenAFS 是对 Network File System (NFS) 的成熟的替代方案，它能适应大量的用户，并能减轻管理的痛苦。它是用于局域网和广域网来分享档案和资源，并不适合柯达云存储的应用场景。

（9）FastDFS

FastDFS 是一个轻量级的开源分布式文件系统，FastDFS 主要解决了大容量的文件存储和高并发访问的问题，特别适合以文件为载体的在线服务。FastDFS 由 tracker server 和 storage server 两部分构成。tracker server 主要在访问上起负载均衡的作用，是连接 client 和 storage server 的枢纽。storage server 是存储服务器，存储服务器采用了分卷（或分组）的组织方式。存储系统由一个或多个卷组成，一个卷可以由一台或多台存储服务器组成，同一卷中的多台存储服务器相互冗余。

通过对云存储以及分布式文件系统的研究可以发现，当前云存储中采用的分布式文件系统主要是面向海量数据以及大文件存储而设计的文件系统，无法兼顾小文件和大文件的存储性能。另外，主要考虑 PC 端的文件存储，对具有移动互联网属性的终端适应性不强。在当下，移动终端与人的生活结合地更加紧密，因而，设计一款面向移动互联网应用的分布式文件系统是大势所趋。Alpaca 云存储系统存储的文件以文档、图片、软件以及配置文件等为主，FastDFS 开源分布式文件系统非常适合 Alpaca 的应用场景，并且 FastDFS 是开源的文件系统，具有较高的经济效益。但 FastDFS 在热文件缓存、负载均衡、大文件访问等方面性能不佳，使其并不能完全满足 Alpaca 云存储系统的设计需求。通过对 FastDFS 分布式文件系统源代码级别的修改，设计缓存机制管理和优化、负载均衡管理和优化以及大文件存储管理和优化三大模块，实现基于 FastDFS 的 Alpaca 云存储在文件缓存、请求的负载均衡、大文件访问等方面的优良特性。

1.3 研究内容

1.3.1 基于 FastDFS 的云存储文件系统优化机制设计

本课题的优化机制设计包括三个部分：

（1）云存储文件系统缓存区管理优化机制 通过研究和制定包括缓存区开辟前置条件、文件查找流程、缓存区文件淘汰机制、缓存区更新策略等一系列的缓存区管理规则，实现热文件的存取优化。

（2）云存储文件系统的请求负载均衡优化方法 通过发现 FastDFS 负载均衡策略的不足并结合 CPU、内存、磁盘、网络带宽等资源使用率等因子，应用现有的负载均衡算法研究成果，实现动态反馈调整集群的流量分配和

全局负载均衡。

(3) 云存储文件系统的大文件分块策略及文件去重机制：通过发现 FastDFS 文件上传机制的不足，应用现有的文件分块以及数据去重算法研究成果，形成新的文件上传机制，最后使用 tracker server 反馈的 storage server 节点存储块文件。

1.3.2 基于 FastDFS 的云存储文件系统架构及功能模块设计与实现

基于 FastDFS 的基本架构，结合三方面的优化机制，本课题的系统实现内容主要包括三个功能模块：

(1) 缓存机制管理和优化模块：主要通过在 client 与 storage server 之间增加缓存层实现。系统每次执行下载文件任务前，先判断被下载的文件是否在缓存区。如果文件在缓存区，直接从缓存区读取文件；如果文件不在缓存区，则从磁盘读取文件并判断文件是否满足加入缓存区的要求，如果文件满足加入缓存区的要求，则按缓存区管理规则处理文件，如果不满足要求，按照正常逻辑下载文件。另外，系统将定时对缓存区的文件时效性进行检查，并按缓存区管理规则进行管理。

(2) 负载均衡管理和优化模块：主要通过在 tracker server 与 storage server 之间增加新的负载均衡策略实现。新的负载均衡策略根据 CPU 使用率、内存使用率、磁盘使用率、网络带宽使用率等因素对节点进行综合评估，以选择合适的 storage server 用于响应文件储取请求。

(3) 大文件存储管理和优化模块：主要通过在 client 与 tracker server 之间增加大文件分块层实现。在上传大容量文件的时候，先判断文件大小是否超过系统设定的阈值，如果超过阈值，则把文件进行分块储存，否则，按原 FastDFS 上传文件的逻辑上传文件。另一方面，客户端对文件进行分块后，将运用重复数据删除技术对源数据消重，然后再执行文件上传操作，这将在一定程度上解决 FastDFS 大文件存储性能不佳的问题。

1.3.3 系统验证与实现

Alpaca 云存储系统是基于 FastDFS 设计的一款适用于移动互联网环境的分布式文件系统，相较于 FastDFS，Alpaca 云存储系统主要从缓存、负载均衡以及大文件存储三个方面着手，在 Storage Server 与 Client 之间、Tracker Server 与 Storage Server 之间以及 Tracker Server 与 Client 之间，分别设计了缓存机制管理和优化、负载均衡管理和优化以及大文件存储管理和优化三大模块。最后在 Linux 环境下实现本系统，然后运用 Alpaca 云存储系统的客户端对本系统的性能与功能进行测试，验证相关功能的实现。

1.4 论文组织结构

第一章为绪论，主要讨论了研究课题的背景、分布式文件系统在国内外的发展现状和发展趋势，以及本文研究的主要内容和意义。

第二章为相关技术基础，首先介绍了 FastDFS 体系结构及其特征，从宏观上把握性能优化设计的方向。然后分析了 FastDFS 基本功能的特性，从细节上把握 FastDFS 的优缺点。最后讨论了下文将要使用的技术，为论文的进一步研究奠定技术基础。

第三章为 Alpaca 云存储系统需求分析，通过分析当前移动互联网的特性以及云存储发展趋势，发掘了当前云

存储用户的痛点，提出了以移动互联网为中心的需求分析，拟从缓存、负载均衡以及大文件存储三个方面的性能优化着手，对 FastDFS 系统进行优化设计与实现，研发一款适用于移动互联网的云存储系统。

第四章为 Alpaca 云存储系统的设计与实现，根据第三章的需求分析，详细设计了缓存机制管理和优化模块、负载均衡管理和优化模块以及大文件存储管理和优化模块三大模块，最后并将其实现到系统中间。

第五章为 Alpaca 云存储系统测试，通过对系统的功能和性能进行测试，说明已经实现的系统能够满足需求。

第六章为总结和展望，总结了本文中系统的设计和实现的过程，对使用的技术进行说明，并说明论文中以后需要继续研究的方向。

第二章 相关技术基础

FastDFS 是由 client、tracker server 和 storage server 三个角色组成, 这种由三角色组成的系统架构, 以轻量级、独特的分组方式以及同组服务器地位完全对等等特色, 使 FastDFS 具有强大的市场竞争力与可持续发展的生命力。与业界其他分布式文件系统相比较而言, FastDFS 分布式文件系统特别适合互联网应用, 它解决了一般文件系统的通用问题如冗余备份、负载均衡等机制, 尤其适用于为以文件数据为载体的在线应用提供存储服务, 而且在可用性和性能上有很好的表现。然而, 在应对移动互联网环境时, FastDFS 性能表现不佳, 需要对其在多个方面进行优化。

FastDFS 优化过程中涉及到了许多关键技术, 主要包括缓存技术、负载均衡技术和重复数据删除及大文件分块技术。缓存技术是为了协调吞吐速度相差很大的设备之间数据传送而采用的技术, 通常是为了缓和 CPU 和 I/O 设备速度不匹配的矛盾。众所周知, 在当前互联网网络环境中常常会出现大量的热数据, 大量用户频繁使用这些热数据时, 对系统频繁的 I/O 请求会给文件系统造成极大的负载压力, 而缓存技术就是一定程度上解决这些矛盾的有效手段。负载均衡技术是一种廉价有效透明的技术, 用于加强网络数据处理能力、提高网络的灵活性和可用性。FastDFS 分布式文件系统只有轮询、选择最大剩余空间、指定服务器等几种简单的静态负载均衡方式, 在 Alpaca 云存储系统中引入动态负载均衡机制, 将大大提高存储系统的吞吐量以及可用性。重复数据删除技术主要是为了缓解存储系统的空间快速增长问题。使用基于源端的重复数据删除技术, 可以减少在网络中传输的数据量, 进而降低能量消耗和网络成本, 并为数据冗余备份节省大量网络带宽。

2.1 FastDFS 体系结构

FastDFS 是一个开源的轻量级分布式文件系统, 由跟踪服务器 (tracker server)、存储服务器 (storage server) 和客户端 (client) 三个部分组成, 它对文件进行管理, 功能包括 文件存储、文件同步、文件访问 (文件上传、文件下载) 等, 解决了大容量存储和负载均衡的问题。特别适合以文件为载体的在线服务, 如相册网站、视频网站等等。其系统结构如图 2-1 所示。

FastDFS 是一款类 mogileFS 的开源分布式文件系统, 它摒弃了 mogileFS 中存储文件信息的 MySQL DB 角色, 使数据库不再成为分布式文件系统的性能瓶颈。它用纯 C 语言实现, 支持 Linux、FreeBSD 等 UNIX 系统。它只能通过专有 API 对文件进行存取访问, 不支持 POSIX 接口方式, 不能 mount 使用。准确地讲, Google FS 以及 FastDFS、mogileFS、HDFS、TFS 等类 Google FS 都不是系统级的分布式文件系统, 而是应用级的分布式文件存储服务。

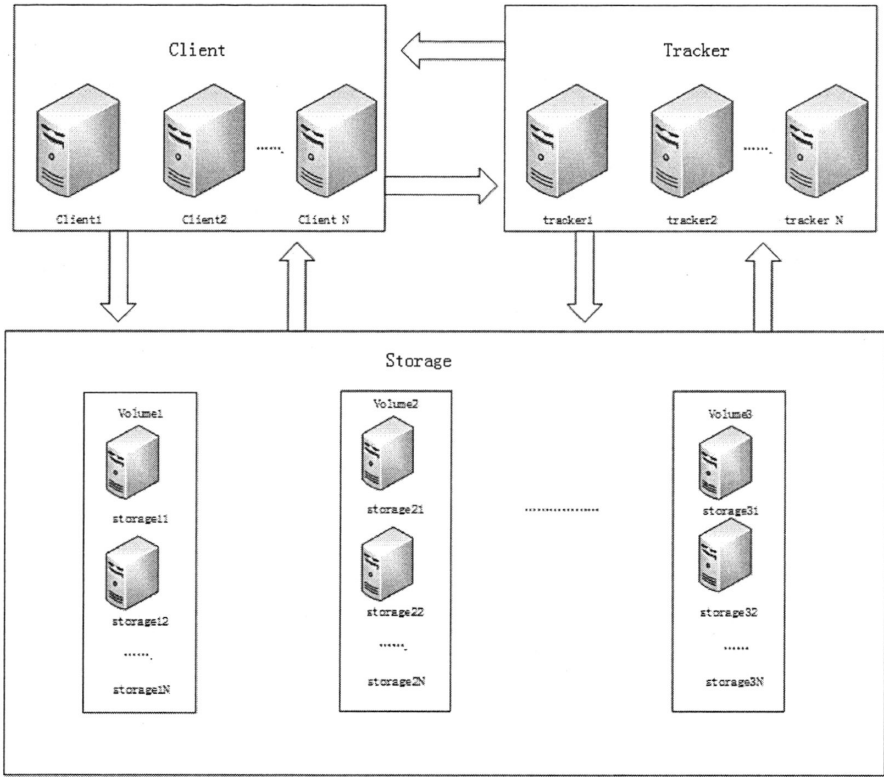


图 2.1 FastDFS 系统架构

2.1.1 FastDFS 中三个基本角色

FastDFS 是一款轻量级的分布式文件系统，充分考虑了冗余备份、负载均衡、线性扩容等机制，尤其适用于为以文件数据为载体的互联网应用提供存储服务。FastDFS 的总体架构图如图 2.1 所示，FastDFS 包含 client、tracker server、storage server 三个角色，Client 负责发生文件操作请求，实际上它也是一台服务器。tracker server 是 FastDFS 的协调者，负责管理所有的 storage server，每个 storage server 在启动后会连接 tracker server，并且会周期性地告知 tracker server 自己所属的 group、文件同步等信息。另外，tracker server 会根据 storage server 反馈的信息建立 group-storage server list 的映射表。Storage server 以组（卷，group 或 volume）为单位组织，一个 group 内包含多台 storage 机器，组中的多台 storage 机器起到了冗余备份和负载均衡的作用，存储空间以 group 内容量最小的 storage 为准。

2.1.2 FastDFS 三大特色

FastDFS 分布式文件系统特别适合互联网应用，它解决了一般文件系统的通用问题，如冗余备份、负载均衡等，而且在可用性和性能上有很好的表现。相比与其他分布式文件系统，FastDFS 的 storage server 的轻量级、独特的分组方式以及同组服务器地位完全对等等是其最大的特色。

a. 轻量级

FastDFS 只有三个角色。client、tracker server 和 storage server。首先，tracker server 作为中心结点集群，其主要作用是负载均衡和调度。tracker server 在内存中记录分组和 storage server 的状态等信息，不记录文件索引信息，占用的内存量很少。其次，storage server 在其他文件系统中通常称作 Trunk server 或 Data server，storage server 直接利用 OS 的文件系统存储文件。另外，客户端上传文件时，文件 ID 不是由客户端指定，而是由 storage server 生成后返回给客户端的。文件 ID 中包含了组名、文件相对路径和文件名，storage server 可以根据文件 ID 直接定位到文件。由以上三点可知，FastDFS 一款比较轻量级的分布式文件系统。

b. 分组方式

FastDFS 采用了分组存储方式。一个组由一台或多台存储服务器组成，同组内的多台 storage server 之间是互备关系，同组存储服务器上的文件是完全一致的。文件上传、下载、删除等操作可以在组内任意一台 storage server 上进行。

采用分组存储方式的好处是灵活、高可用。当一个分组的存储服务器遇到存储瓶颈时，可以在该组内增加存储服务器来纵向扩充服务能力。当整个系统遇到存储瓶颈时，可以增加组来横向扩充服务能力。因而采用这样的分组存储方式，极大地增加了 FastDFS 的可用性。

c. 对等结构

传统的 Master-Slave 结构中的 Master 是单点，写操作仅针对 Master。如果 Master 失效，需要将 Slave 提升为 Master，实现逻辑会比较复杂。而 FastDFS 集群中的 tracker server 可以有多台，tracker server 和 storage server 均不存在单点问题。tracker server 之间是对等关系，组内的 storage server 之间也是对等关系。

2.1.3 小文件存储及文件命名策略

FastDFS 是一个应用级分布式文件存储服务，主要用于大中型网站存储资源文件。FastDFS 具有轻量级、支持高并发访问、负载均衡、可扩展等优点。而 FastDFS 较大的亮点就是对小文件的存储性能较好，这主要来自于其文件名策略。

a. 小文件存储性能优化

小文件的性能瓶颈主要来自于对元数据服务器（如 FastDFS 中的 TrackerServer 或 TFS 中的 NameServer）的访问，因为当文件本身大小很小时，元数据存储所占空间与文件内容存储所占空间的比例就变得较大，访问元数据所消耗资源与访问文件内容所消耗资源的比例也变得较大。因此，通常对小文件存储的优化方法主要有两大类思路：一是减少访问元数据的次数，比如 Cache 预取，二是减少元数据所占的存储空间，比如 FastDFS 使用的文件名策略。

b. FastDFS 文件名策略

在 FastDFS 中，客户端上传文件时，文件 ID 不是由客户端指定，而是由 Storage server 生成后返回给客户端的。当文件存储到某个子目录后，即认为该文件存储成功，接下来会为该文件生成一个文件名，文件名由 group、存储目录、两级子目录、fileid、文件后缀名（由客户端指定，主要用于区分文件类型）拼接而成，Storage server 可以通过对文件 ID 解析直接定位到文件存储位置。因此 FastDFS 集群中根本不需要存储文件索引信息，这是 FastDFS 比较轻量级的一个例证。其他文件系统则需要存储文件索引信息，这样的角色通常称作 NameServer。其中 mogileFS 采用 MySQL 数据库来存储文件索引以及系统相关的信息，其局限性显而易见，MySQL 将成为整个系统的瓶颈。

文件名格式如图 2.2 所示。

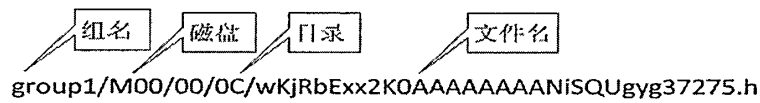


图 2.2 文件名格式

2.1.4 负载均衡方式

FastDFS 是一个分布式文件系统，它是基于计算机集群技术实现的。在计算机集群中，负载均衡是整个系统必不可少的一个功能，负载均衡就是通过某种负载技术使负载能够均匀地分配到每一台服务器上，然后由收到请求的服务器独立地完成任务。FastDFS 系统中只有轮询、选择最大剩余空间、指定服务器等几种简单的负载均衡方式，这些都是属于静态负载均衡算法，在生产环境的流量大小相对稳定以及并发量并不高的情况下，FastDFS 能较好地满足存储需求。但是，在大规模集群中的表现往往差强人意，因为 FastDFS 的负载均衡算法具有无法考虑集群的真实负载，算法效率低下，容易使性能较低的计算机节点成为整个系统的瓶颈等缺点。

2.1.5 文件上传

- 1 Client 询问 Tracker server 上传到的 Storage server;
- 2 Tracker server 返回一台可用的 Storage server，返回的数据为该 Storage server 的 IP 地址和端口，
- 3 Client 直接和该 Storage server 建立连接，进行文件上传，Storage server 返回新生成的文件 ID，文件上传结束，文件上传流程如图 2.2 所示。

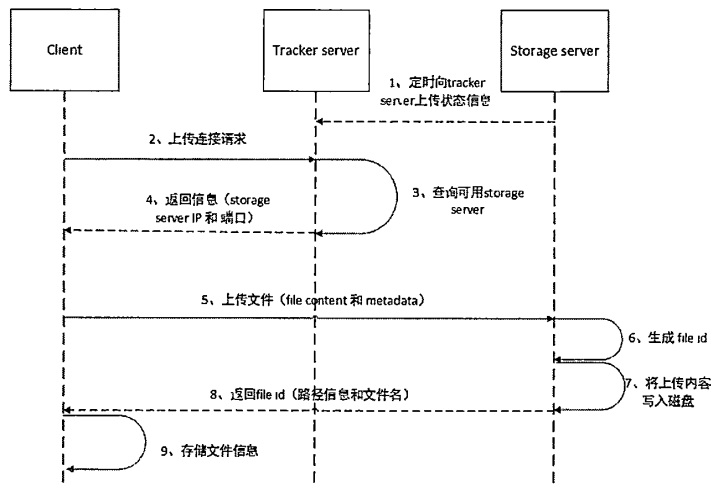


图 2.2 文件上传流程

2.1.6 文件下载

- 1 Client 询问 Tracker server 可以下载指定文件的 Storage server，参数为文件 ID（包含组名和文件名）；
- 2 Tracker server 返回一台可用的 Storage server，

3 Client 直接和该 Storage server 建立连接，完成文件下载。

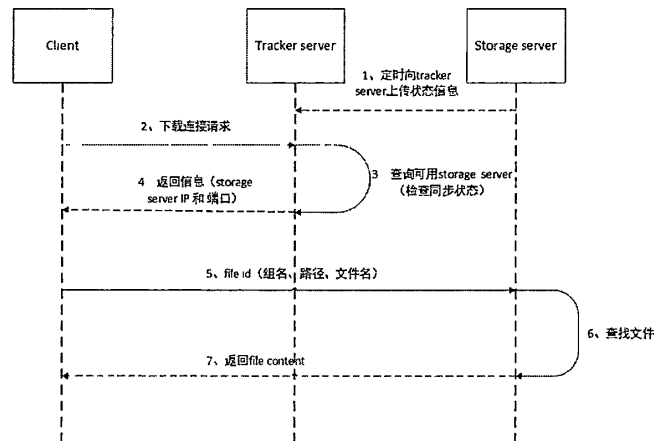


图 23 文件下载流程

2.2 关键技术分析

2.2.1 缓存技术

随着计算机硬件技术的不断发展，CPU 运算速度、随机访问内存与磁盘的容量等都有了长足的进步，而磁盘等 I/O 设备由于受限于本身机械构件的特点，发展速度相对慢了很多。CPU 运算速度与 I/O 系统访问速度的越来越大的差距形成了“ I/O 墙”。因此，I/O 系统成为了整个存储系统性能的主要瓶颈。

为了缓解 I/O 设备对系统整体性能的负面影响，科研人员提出了各种各样的缓存技术。我们注意到，在大型分布式系统中间，系统节点往往会分配较大的物理内存，而在系统运行的大部分时间里，各计算节点经常会闲置较大的物理内存空间，如果我们能把这些闲置的内存空间利用起来，把计算机中被频繁访问的热数据缓存到内存中，屏蔽访问这部分热数据的 I/O 开销，将能大大提升系统整体性能。

缓存是弥补 CPU 与存储系统之间巨大速度差异的有效手段。但受成本和制作工艺限制，缓存与存储系统间的容量差异巨大，因此，如何最大效率地利用内存缓存以及深入挖掘内存缓存的潜在性能，是提升分布式系统整体性能的一个有效技术途径。在硬件条件相对固定的情况下，缓存算法是提升缓存性能的一个有效技术手段，缓存算法能高效地组织缓存在内存中的进入与退出，从而提升内存缓存空间的利用率，达到提升系统整体性能的目的。常见缓存算法包括 LFU、LRU、FIFO 等。

(1) LFU

LFU 算法是根据缓存的历史访问频率来进行缓存置换，如图 24 所示，它需要动态地统计每个缓存数据被命中的次数，然后在进行排序后替换次数较小的缓存。但是，该算法存在“缓存污染”问题，比如，当某个缓存数据的计数值很大时，即使该数据已经过时，此数据也会一直被保存在缓存中，从而导致缓存效率降低。

(2) LRU

LRU 算法是基于数据访问时存在的时间局部性原理，将短时间内最不常被访问的缓存数据替换出缓存，最常被访问的缓存置换到最容易被访问到的位置。如图 2 5 所示，其优点是当数据在短时间内具有明显的时间局部性特征时，LRU 算法有比较出色的表现；反之，如果数据的访问在短时间内没有明显的时间局部性特征，其算法的效能就不太明显。

(3) FIFO

在 FIFO Cache 设计中，核心原则就是：如果一个数据最先进入缓存中，则应该最早淘汰掉。它的优点是比较容易实现，但是，如图 2 6 所示，对热度较高的数据命中不高，因为最先调入内存的数据，很可能也是经常要使用的数据，这个数据将会被最先淘汰。

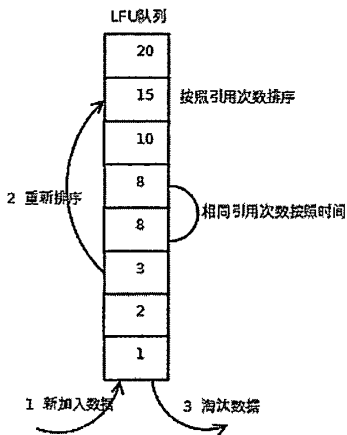


图 2 4 LRU 算法

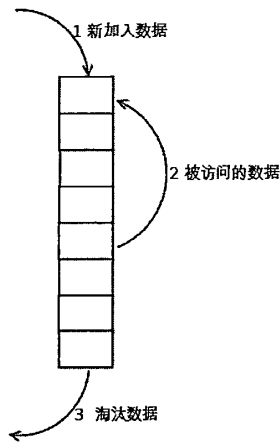


图 2 5 LRU 算法

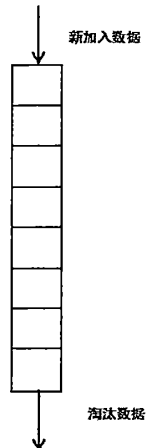


图 2 6 FIFO 算法

2.2.1.1 FastDFS 缓存技术现状

FastDFS 的分组方式、server 地位的完全对等结构以及文件 ID 命名策略等是其最大的特色，这些特色使 FastDFS 整个系统具有简洁高效、轻量级以及可拓展性强等特性，同时也是 FastDFS 的设计初衷。正是基于这些设计初衷，FastDFS 舍弃了一些非核心功能，比如文件系统的缓存模块。但是 FastDFS 的开源为 FastDFS 的可持续发展提供了可能，各开发者在遵循其开源许可证条件下，可自由地扩展 FastDFS 的功能。因而，Alpaca 云存储系统基于客户需求，为了解决大量客户在访问热文件时，系统资源消耗过大、高延迟以及数据传输速度慢等缺陷，本文为 FastDFS 设计了为热文件提供缓存功能的缓存模块，使 FastDFS 能应对当前移动互联网环境下高频次的文件存取需求。

2.2.2 负载均衡技术

当前，互联网面临着用户量级快速增长以及复杂多样的业务形式，访问量的爆炸式增长使很多服务器不堪重负，不能及时处理用户请求，导致用户等待时间过长，大大降低了服务水平，因此，如何建立高可用、高性能的网络服务来满足不断增长的负载需求成为亟待解决的问题。企业在为用户提供网络服务的过程中，网络中配置较差的服务器便成为了制约网络服务质量的瓶颈，服务提供商往往有两种可选的解决方案。一种是不断提高服务器的性能，例如不断使用高性能服务器替换低性能的服务器。在企业成本预算的制约下，很难大批量地更换服务器

来满足企业业务发展的需求,如图 2.7 所示,一种合理可行的解决方案便是采用服务器集群加负载均衡技术。服务器集群是指把多台服务器通过高速通信链路连接起来,提供了一种高性能、低成本的网络服务解决方案,以其可扩展性强、管理维护方便、处理能力突出等优点得到了国内外研究机构的关注和迅速发展。对外而言,这些服务器集群看来就像传统的单个服务器在工作。对内而言,服务器集群技术可以在付出成本较低的情况下获得在性能、可靠性、灵活性等方面较高的收益。随着互联网的发展,在面对大量用户访问、高并发请求方面,提高集群性能基本解决方法集中为使用高性能的服务器、数据库与 Web 服务器,采用高效率的编成语言。但是这些并不能处理问题的根本,解决服务器集群的负载均衡问题是提高集群性能的根本所在。

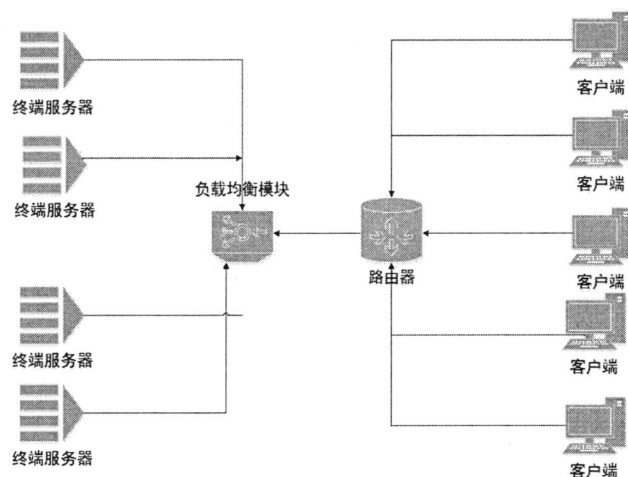


图 2.7 具有负载均衡的服务器集群

负载均衡技术是建立在现有网络的基础上,提供一种廉价、有效、透明的方法扩展网络设备和服务器带宽、增加服务器集群吞吐量、加强网络数据处理能力、提高网络的灵活性和可用性。一般来说,负载均衡要达到的目标是,使各台计算机之间的负载基本均衡,各节点内的负载基本相等。其按照实现方法的不同通常分为硬件实现和软件实现两种^[13]。硬件负载均衡解决方案是直接安装在服务器和外部网络间安装负载均衡设备,这种设备通常称之为负载均衡器,由专门的设备完成负载均衡任务,它独立于操作系统,能使服务器集群整体性能得到大量提高,但价格昂贵;虽然硬件负载均衡解决方案集成多种负载均衡算法,但是灵活性不强,不支持许多性能优化的负载均衡算法以及更复杂的应用协议。此外,它只是通过对网络层进行监测来判断数据流量大小,无法有效地动态掌控服务器和应用的负载状况。然而,通过软件的方式能够更好地实现根据系统与应用的状况来分配负载,灵活性好,性价比高,并且易于根据实际情况更新负载均衡策略。但基于软件方式的负载均衡会对服务器的性能造成一定的影响,所以对算法的复杂度要求较高。

在软件实现中,实现集群负载均衡的核心部件是负载均衡算法,在做负载均衡算法设计时,主要从以下几个维度实现算法:

- 1) 依序 Round Robin
- 2) 比重 Weighted Round Robin
- 3) 流量比例 Traffic
- 4) 使用者端 User

- 5) 应用类别 Application
- 6) 联机数量 Session
- 7) 服务类别 Service
- 8) 自动分配 Auto Mode

根据这些设计方式的不同使其主要分成静态均衡算法与动态均衡算法两类^[14-16]。静态均衡算法只是利用集群的信息以固定的方式来分配任务，而不考虑集群中节点的实际负载状况，因而已无法适应当前复杂多变的互联网环境。动态均衡算法则通过实时采集集群的运行信息来评估集群中节点的负载状态，进而动态调整请求任务的分配，避免系统长时间运行下任务的分配发生倾斜。实验表明，动态均衡算法相较于静态均衡算法具有更好的性能，即使在极端的情况中，动态均衡算法仍然能取得比较理想的性能。一般情况下，相对于较静态均衡算法而言，动态算法能获得 30%~40% 的性能改善^[17]，随着对动态负载均衡算法的研究，动态均衡技术的应用广度必将大大超越静态均衡技术。

2.2.2.1 FastDFS 负载均衡技术现状

FastDFS 是基于计算机集群技术实现的分布式文件系统，如图 2 8 FastDFS 配置文件所示，FastDFS 系统中只有轮询、选择最大剩余空间、指定服务器等几种简单的负载均衡方式，这些负载均衡方式在应对高负载、高并发的移动互联网业务需求时往往差强人意，因为服务器的真实负载情况与 CPU 利用率、磁盘 I/O 性能、内存利用率以及网络吞吐量等多方面因素有关，仅凭轮询、选择最大剩余空间、指定服务器等几种简单的负载均衡方式无法有效地反应集群系统的负载。特别是在大规模计算机集群环境中，FastDFS 简单的负载均衡方式对某些节点带有偏向性，这将导致部分节点率先因资源耗尽而失去服务能力。因此，应用现有的动态负载均衡算法研究成果，实现动态调整集群的流量分配和全局负载均衡是 FastDFS 的优化目标之一。

```
# the method of selecting group to upload files
# 0 round robin
# 1 specify group
# 2 load balance, select the max free space group to upload file
store_lookup=2

# which group to upload file
# when store_lookup set to 1, must set store_group to the group name
store_group=group2

# which storage server to upload file
# 0 round robin (default)
# 1 the first server order by ip address
# 2 the first server order by priority (the minimal)
store_server=0

# which path(means disk or mount point) of the storage server to upload file
# 0 round robin
# 2 load balance, select the max free space path to upload file
store_path=0

# which storage server to download file
# 0 round robin (default)
# 1 the source storage server which the current file uploaded to
download_server=0
```

图 2 8 FastDFS 配置文件

2.2.3 重复数据删除及大文件分块技术

随着互联网数据量的爆炸式增长，数据占用的存储空间越来越大；在过去数 10 年里，许多存储系统提供的存储容量从数十 GB 发展到数百 TB，甚至到数 PB。随着数据的指数级增长，企业快速备份和恢复的时间点越来越多，导致管理数据的成本快速增加。研究发现，存储系统中所保存的数据中高达 60% 是冗余数据，而且随着时间的推移冗余数据越来越多。为了缓解存储系统容量快速增长的问题，缩减数据占用空间，降低存储成本，最大程

度地利用存储系统资源,重复数据删除技术已成为一个热门的研究课题。一方面,重复数据删除技术可以消除分布在存储系统中的相同文件或者数据块,以达到对存储空间利用率进行优化的目的。另一方面,利用重复数据删除技术可以减少客户端和服务端之间传输的数据量,并为数据冗余备份节省大量网络带宽,进而降低能量消耗和网络成本。

随着重复数据删除技术的快速发展,该技术大量应用于归档系统和云存储系统中,该系统中的重复数据删除模块对数据内容进行分析与比对,查找出冗余数据内容,并将其元数据反馈给存储系统,最后将不重复的数据存储到系统中。重复数据删除技术主要分为以下两大类:

(1) 相同数据检测技术,相同数据主要包括文件级别和数据块级别两个层次,主要是通过 hash 技术进行数据去重。完全文件检测(whole file detection, 简称 WFD)技术是一种基于文件级别的数据去重方法,存储系统中存有每个文件的唯一 hash 值(一般是 MD5 值或者 SHA-1 值),每当客户端向存储系统发出文件存储请求时,系统都会先检测当前将要被存储的文件的 hash 值,并与存储系统的 hash 库进行比对,进而决定是否把该文件以及 hash 值存进存储系统,从而保证文件在整个存储系统中的唯一性。但是这种方式依然会使存储系统中出现很大数据冗余,因为两个文件内部还是可能出现很多相同的数据。对此,研究人员提出了更细粒度的数据去重方法,如图 2.9 所示,也就是基于数据块级别的相同数据检测技术。数据块级别的相同数据检测主要通过固定分块(fixed-sized partition, 简称 FSP)检测技术、可变分块(content-defined chunking, 简称 CDC)检测技术、滑动块(sliding block)技术等重复数据的查找与删除。数据块级重复数据删除技术是根据一定的策略将文件分块,删除在存储系统中检索出的具有相同内容的数据块。毫无疑问,数据块级重复数据删除技术可以提供较高的数据去重效率,但是实现起来相对复杂,开销也大,在存储系统中要使用何种重复数据删除技术,要因地制宜。

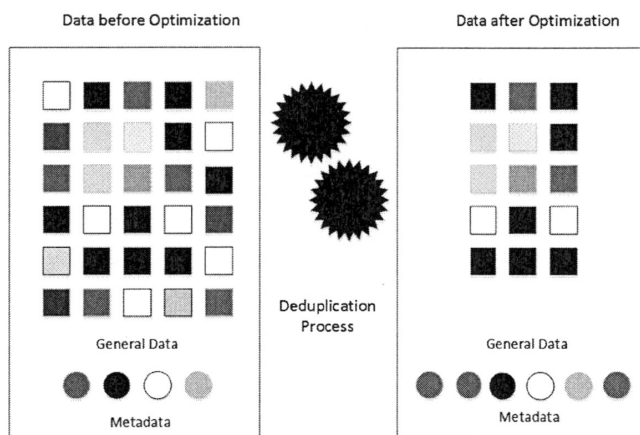


图 2.9 数据块级重复数据删除技术

(2) 相似数据检测和编码技术,利用数据自身的相似性特点,通过 shingle 技术、bloom filter 技术和模式匹配技术挖掘出相同数据检测技术不能识别的重复数据。对相似数据采用 delta 技术进行编码并最小化压缩相似数据,以进一步缩减存储空间和网络带宽的占用。由于该项技术可靠性有待提高,应用到实际中也有待研究,因此本文主要研究和应用相同数据检测技术,这种方法具有更广泛的适用性。

2.2.3.1 FastDFS 重复数据删除及大文件分块技术现状

FastDFS 是一个开源的轻量级分布式文件系统，其专门为互联网应用而定制，特别适合以文件为载体的在线服务，如相册网站、视频网站，电子文档存取网站等。出于降低带宽和存储成本的考虑，网站通常都会限制用户上传的文件大小，通常文件大小在 500MB 之内，因此，对于互联网服务来说，重复数据删除以及大文件没有实用性，因为既没有给系统带来实际增益，又增加了系统的复杂性。与支持文件分块的分布式文件系统相比，FastDFS 更加简洁高效，更能胜任互联网业务的需求。然而，在应对大文件存取需求时具有以下几点不足，第一，在大文件完全同步到其他 storage server 之前，其他 storage server 无法对该文件提供存取服务。第二，由于文件过大，在文件上传与下载过程中对于网络的稳定性与可靠性要求比较高。第三，如果在某台 storage server 宕机的情况下，该 storage server 必须同步整个文件后才能对外提供服务。使用大文件分块技术后，只需要同组其他机器同步过来一个文件块的情况下，本机就可对外提供该文件的下载服务，提高了系统的可用性。

2.3 本章小结

本文首先介绍了 FastDFS 的基本体系结构及这种结构所带来的优势，接着详细介绍了 FastDFS 各类功能特性以及部分工作流程，并说明了这些特性相较于其他分布式文件系统的优势。最后，详细阐述了系统优化中所有涉及到的技术，并根据当前互联网环境来分析选择这些技术来支撑系统优化的原因。

第三章 系统需求分析

Alpaca 云存储系统是基于 FastDFS 分布式文件系统而设计的, FastDFS 分布式文件系统特别适合互联网应用, 它解决了一般文件系统的通用问题如冗余备份、负载均衡等, 而且在可用性和性能上有很好的表现。但是, 由于云存储的用户群体庞大且网络状况复杂, 用户对于文件存储尤其是大文件存储的需求日益增强, FastDFS 分布式文件系统作为 Alpaca 云存储系统的核心底层构件, 已无法满足全球 Alpaca 云存储系统用户对文件存取性能的需求。首先, FastDFS 在大量客户在访问热文件时, 系统存在资源消耗过大、高延迟以及数据传输速度慢等缺陷; 其次, 在大规模集群应用场景中, 因为 FastDFS 的负载均衡算法具有无法考虑集群的真实负载, 算法效率低下等缺点, 容易使性能较低的计算机节点成为整个系统的瓶颈, 最后, FastDFS 分布式文件系统出于设计精简的考虑, 未对文件做分块存储及数据块去重处理, 这导致用户会耗费大量不必要的文件传输时间与流量, 同时也影响 Alpaca 云存储系统的文件冗余备份性能。基于以上问题, 基于 FastDFS 设计的 Alpaca 云存储系统需要设计三大模块 缓存机制管理和优化模块、负载均衡管理和优化模块以及大文件存储管理和优化模块, 使其满足 Alpaca 云存储系统的全球化应用场景。

3.1 系统总体功能需求分析

目前, 柯达服务于全球 100 多个国家的用户, 柯达在为全球客户提供智能化打印服务过程中, 产生了海量的设备色彩特性文件、DeviceLink 文件, 商业印刷中的包装、封面、LOGO 以及版式等设计稿件, 个性化影像业务和景点影像解决方案形成的影像文档等各类文件。柯达面对当前 PB 级的海量各类文件存储需求, 传统的 SAN 或 NAS 在容量和性能的扩展上会存在瓶颈, 一旦出现瓶颈, 企业要进行存储设备的更新升级, 必将面临高额的成本支出。云存储的兴起正在颠覆现有的网络存储架构。如何利用云计算、云存储技术高效地整合这些文件, 提升柯达的服务品质, 是柯达急需解决的一大难题。为此, 柯达研发了基于 FastDFS 的 Alpaca 云存储系统, Alpaca 云存储系统的研发和使用, 将有利于柯达整合全球客户的印刷解决方案, 并通过 Alpaca 云存储客户端便捷地分享给全球的柯达用户, 提升智能化打印服务的品质。本选题依据正是 Alpaca 云存储系统的实际项目需求以及市场调研的结果。

移动互联网的快速发展, 加上个人云存储服务等以私有云为基础的数据存储服务快速兴起, 推动了面向移动互联网数据的云同步和云存储业务快速增长, 使得移动网络中传输的数据量呈现爆炸式增长形势, 给网络存储空间和网络带宽带来了巨大的压力。该系统建成后将为全球柯达用户提供基于移动互联网的灵活性好、安全性高以及按需定制的新一代存储服务。从短期和长期来看, Alpaca 云存储服务将为用户减少大量生产成本, 回避使用本地存储服务所产生的一些不良后果。另外, 生产资料的云端共享将促进客户间的学习和交流, 改善生产工艺, 提升整个智能化打印行业水平。最后, Alpaca 云存储服务将能最大限度地保持柯达产品的用户粘性, 保持客户对柯达产品的忠诚度, 提升柯达产品的全球市场份额。

3.2 业务场景功能需求分析

3.2.1 缓存机制管理和优化模块

在移动互联网环境中，基于 FastDFS 性能优化设计的 Alapca 云存储系统中文件的读取相对传统互联网应用要频繁许多，频繁的 I/O 请求会在 Alapca 云存储系统中产生许多热文件，对于被高频次访问的热文件，FastDFS 分布式文件系统每次均从磁盘读取。与 CPU 和内存速度的飞快发展相比，磁盘 I/O 速度在近些年并没有得到显著提升，低下的 I/O 性能严重影响着各类互联网应用的用户体验，尤其像 Alapca 云存储系统这类 I/O 密集型应用。针对此类问题，在软件解决范围内主要的方案是减少不必要的磁盘 I/O 操作。因此，为了提升 Alapca 云存储系统对热文件的访问性能，减少不必要磁盘 I/O 操作比较有效的方式就是建立合理的 I/O 缓存机制。I/O 缓存机制就是将数据从低速率的介质上缓存到高速率的介质上，系统在处理数据访问请求的时候，优先访问高速率介质。如果在高速率介质上没有找到请求的数据，则再从低速率介质上读取数据。如此，用户从云存储系统读取文件的速率将得到大大提升。

用户和应用程序访问系统中的数据具有局部性特点，数据访问的局部性分为时间局部性和空间局部性。空间局部性是指刚才被访问的数据所在相邻位置的数据也可能被一并访问，空间局部性原理使得存储系统可以使用预读取技术来制定缓存策略。时间局部性原理是指刚才被访问的数据能在短时间被再次访问，很多存储系统的缓存策略就是依照时间局部性原理来制定的。Alapca 云存储系统是一款应用于用户量庞大的在移动互联网环境中，文件读取具有较大的随机性，因此更适宜使用时间局部性原理来制定内存缓存策略。另外，缓存在很多方面都起着举足轻重的作用，并且它以各种各样的形式存在不同的硬件里面，其中有内存缓存，硬盘和软盘缓存，软件磁盘缓存，还有页面缓存等更多。在实际生产环境中，存储节点往往有较大的内存容量被闲置。利用闲置内存以及数据时间局部性原理来制定 Alapca 云存储系统中的 I/O 缓存机制，这将在一定程度上提升存储系统的可用性，实现了热文件的存取优化。

3.2.2 负载均衡管理和优化模块

云存储技术是通过计算机集群技术来实现的，计算机集群技术将一组相互独立的个体联合起来作为一个整体向用户提供某类服务，此类计算机集群具有高性能、高可用性以及高伸缩性。由于集群内部的计算机节点不一定是完全一致的，总是存在 CPU 性能、I/O 性能以及内存大小等差异，如何合理地分配集群的业务量，避免某些计算机节点出现过载或者某些计算机节点能力未得到充分发挥等情况，这就成了集群系统一个非常重要的问题。计算机集群负载均衡技术的目的是在不改变现有系统架构的前提下，提供与计算机节点个数成比例的处理能力，主要是通过合理分配集群中各节点的负载来达到提升集群系统整体性能的目的，如提高集群吞吐量、减少响应时间、避免单节点故障、确保高性能服务器的资源得到充分利用等。

FastDFS 是一个分布式文件系统，它是基于计算机集群技术实现的，FastDFS 系统中对于 group 的选择只有三种负载均衡方式，一是轮询，它的原理是把来自用户的请求轮流分配给内部的 group。从 group1 开始，直到 group N，然后重新开始循环；二是指定 storage 组，也就是在配置文件中指定某个 group 来响应服务；三是平衡负载，也就是选择具有最大剩余存储空间的 group 响应服务。这些都是属于静态负载均衡算法，静态负载均衡算法在大

规模集群时往往差强人意,无法满足 Alpaca 云储存系统的全球化应用场景对负载均衡的需求。主要有以下几点不足:

(1) FastDFS 的负载均衡算法没有考虑集群的真实负载。因为服务器的真实负载情况与 CPU 利用率、磁盘 I/O 性能、内存利用率以及网络吞吐量等多方面因素有关, FastDFS 仅凭轮询、选择最大剩余空间、指定服务器等几种简单的负载均衡方式无法有效地反应集群系统的负载。针对以上经典负载均衡算法的上述缺陷,我们在选用负载均衡算法时,应该综合多方面因素来描述服务器的负载情况,使服务器的负载状态尽可能客观真实。

(2) FastDFS 的负载均衡算法效率低下。例如 FastDFS 在使用其选择最大剩余空间的负载均衡算法, FastDFS 的 tracker server 每次收到请求是都会往拥有最大剩余空间的 storage server 分配任务,这将导致单个或者某些 storage server 在某个时间段内处理任务量急剧上升,这将会耗费大量系统资源,增加用户等待时间。并且,从集群的一个较长服务时间跨度来看,系统任务量的分配总是往某个或者某些 storage server 偏向,并没有合理地往有较大剩余处理能力的 storage server 分配,这将导致系统总是处于较长的反应时间。如果将 storage server 按照负载状况分为轻载节点和非轻载节点,以从高到低的优先级按相应的负载均衡策略提供服务,并检测 FastDFS 系统中是否有 storage server 负载过重,这将提高算法的运行效率,调度更加精细。

(3) FastDFS 容易使性能较低的 storage server 成为整个系统的瓶颈。轮询、选择最大剩余空间等简单负载均衡算法在使用过程中会性能较低的 storage server 最先资源耗尽(如磁盘资源耗尽),这将是部分 storage server 率先停止服务,这将降低整个系统的稳定性、高可用性。因此选用负载均衡算法的时候,应该尽量考虑到各个 storage server 资源的保有量以及剩余处理能力,让各个 storage server 资源按一定的比例被消耗,确保整个系统没有 storage server 因为资源快速消耗殆尽而退出服务。

通过发现 FastDFS 负载均衡策略的不足并结合 CPU、内存、磁盘、网络带宽等资源使用率等因子, Alpaca 云存储系统应用了现有的动态负载均衡算法研究成果,实现动态反馈调整集群的流量分配和全局负载均衡。有实验表明,负载均衡算法可以取得比静态负载均衡算法更好的性能,即使在极端的集群负载特性中,动态负载均衡算法仍然能取得比较理想的性能。一般情况下,相对于较静态均衡算法而言,动态算法能获得 30%-40%的性能改善[7]。

3.2.3 大文件存储管理和优化模块

随着移动互联网的发展,个人云存储应用已成为解决个人数据跨平台共享及协调工作的主要方式,通过移动终端获取数据服务变得越来越普及。移动数据服务的普及给 IT 企业的数据备份、迁移以及恢复造成了较大的负面影响,给 IT 企业增加了较高的经营成本。另外,这种数据服务同时也对网络带宽、网络存储设备服务能力以及手机性能提出了较高的要求。所以,如何减少存储空间的冗余数据,最大程度地有效利用移动设备存储空间和减少网络带宽消耗成为了移动云存储领域研究的热点问题之一。

在互联网环境中,云存储系统对于单个大文件的上传、下载以及冗余备份等操作是相当费时的。特别是在 FastDFS 中,当同组 storage server 较多时,大文件的同步过程会占用较多的 CPU 资源,并且短时间内除了源 storage server 之外的 storage server 无法立即对外提供文件存储服务。当大文件所在的 storage server 宕机时,同组其他 storage server 需要花费较长的时间把该文件同步过来,因而本机需要较长时间才能对外提供文件存储服务。如果把文件块作为数据备份的单位,这将会在一定程度上提供数据容错能力和系统可

用性。一方面，为了防止数据块损坏或者机器故障，每一个文件块都被冗余备份到少数几台独立的机器上。另一方面，为了提高文件存储效率，可以利用负载均衡技术选择性能较佳的存储节点提供文件上传和下载服务。

FastDFS 相较于市场同类分布式文件系统而言，在文档、视频以及图片等文件存储方面具有较大的优势，但是 FastDFS 分布式文件系统出于设计精简的考虑，未对文件做数据块去重处理，在文件上传过程中将耗费大量不必要的上传时间以及流量，与此同时，存储系统还将耗费大量的存储空间存储重复数据。使用重复数据删除技术，这将在一定程度上解决这些问题。重复数据删除技术根据重复数据删除操作发生的位置不同可分为目标端重复数据删除和源端重复数据删除。目标端重复数据删除发生在服务器端，这类重复数据删除不受客户端的控制，一般来说，这类重复数据删除的去重速度较快，但是客户端得花费较长的时间和较高的带宽来传输源文件。源端重复数据删除是发生在客户端，将具有唯一性的文件传输到服务器端，这样的好处是降低文件传输时间和节省网络带宽消耗，同时降低了服务器端文件备份与恢复的成本。因此，在移动互联网环境中，使用基于源端的重复数据删除技术能显著提高用户体验，降低 IT 企业的运营成本。Alpaca 云存储系统应用 `fingertdiff` 算法在源端实现大文件进行重复数据删除，然后对进行了重复数据删除的数据文件实行分块存储，这将解决 FastDFS 在大文件存储方面的固有缺陷，使其适应移动互联网生产环境。

3.3 本章小结

本章对系统进行需求分析，介绍了系统的体系架构和功能模块设计，使系统更加清晰明了。从云存储到分布式文件系统的应用，再到做一块适合移动互联网的分布式文件系统所涉及的技术选型，最后从缓存、负载均衡以及大文件存储三个方面着手，对 FastDFS 系统进行优化设计与实现，研发一款适用于移动互联网的云存储系统。

第四章 Alpaca 云存储系统模块设计与实现

基于 FastDFS 分布式文件系统的整体架构特点和系统各个模块之间的内在关联以及 FastDFS 在应用于移动互联网领域时所暴露的问题，提出了详细的 FastDFS 性能优化解决方案，设计了缓存机制管理和优化、负载均衡管理和优化以及大文件存储管理和优化三大模块。缓存机制管理和优化模块是设计在 Storage Server 与 Client 之间，实现对计算过程中各节点的热数据进行缓存，减小磁盘 I/O 开销。负载均衡管理和优化模块是设计在 Tracker Server 与 Storage Server 之间，解决 FastDFS 的静态负载均衡算法在大规模集群时表现不佳的痛点，该模块将为 Alpaca 云存储系统提供基于 CPU 使用率、内存使用率、磁盘使用率、网络带宽使用率等因素的动态负载反馈能力，提升 Alpaca 云存储系统的可靠性。大文件存储管理和优化模块是设计在 Tracker Server 与 Client 之间，解决 FastDFS 应对大文件存储效率不佳的问题。该模块利用 Fingerdiff 算法以及大文件分块技术，从文件块的粒度减少了大文件中的重复数据，并提高了大文件上传与下载的效能。

4.1 开发环境

操作系统： centos 6.5
编程环境： Emacs、KDevelop
开发语言： C

4.2 Alpaca 云存储系统总体框架

基于 FastDFS 的 Alpaca 云存储系统总体框架如图 4.1 所示：

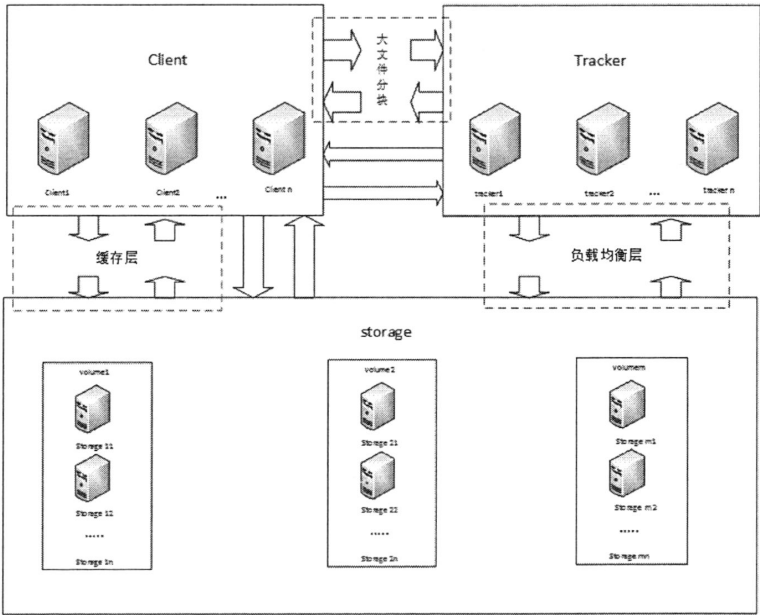


图 4.1 Alpaca 云存储系统总体架构图

Aplaca 云储存系统的分布式文件系统基于 FastDFS 的基本架构, 主要包括跟踪器 (tracker)、存储节点 (storage server) 以及 client 节点三部分。tracker server 是 FastDFS 的协调者, 负责管理所有的 storage server, 每个 storage server 在启动后会连接 tracker server, 并且会周期性地告知 tracker server 自己所属的 group、文件同步等信息。另外, tracker server 会根据 storage server 反馈的信息建立 group--storage server list 的映射表。另外, 客户端和 storage server 访问 tracker server 时, tracker server 扫描内存中的分组和 storage server 信息, 然后给出应答。存储节点 (storage server) 采用分卷 (或分组) 的组织方式, 一个卷可以由一台或多台 storage server 组成, 同一个卷下的多台 storage server 起到了冗余备份和负载均衡的作用, 存储空间以 group 内容量最小的 storage 为准, client 也是一台服务器, 它一端面向用户, 一端面向 tracker server 和 storage server, 为这两者提供具体的业务逻辑处理。

基于 FastDFS 的系统架构, 针对热数据对系统磁盘 I/O 性能产生负面影响的问题, 设计了 client 与 storage server 之间的缓存层, 制定基于文件请求热度的缓存区管理机制, 提高 Aplaca 云储存系统热文件的访问性能, 针对 FastDFS 原有负载均衡方式无法胜任 Alpaca 云存储系统全球化应用场景的问题, 优化了 tracker server 与 storage server 之间的负载均衡层, 引入节点负载状况评价模型以及节点处理能力评价模型评估 storage server 适合响应 client 的请求的概率, 实现 Aplaca 云储存系统的流量分配和全局负载均衡, 针对 FastDFS 对大文件支持不足的问题, 在 client 与 tracker server 间增加大文件处理的模块, 实现文件变长分块算法以及数据去重技术, 提高 Aplaca 云储存系统大文件处理性能。

4.3 缓存机制管理和优化模块

在存储系统缓解 I/O 设备对系统整体性能负面影响的解决方案中, 构建性能良好的缓存模块是其重要的一环, 利用大型分布式系统中服务节点的大容量闲置物理内存, 实现对计算过程中各节点的热数据进行缓存, 屏蔽访问这部分热数据的网络、磁盘 I/O 开销, 是构建缓存模块的重要手段, 这将会大大提升系统整体性能, 同时降低分布式存储系统的访问负载。

Alpaca 云存储系统利用 Storage Server 闲置的物理内存, 在 Storage Server 与 Client 之间增加缓存层, 每次客户端对 Storage Server 上的文件进行操作时, 先去内存中查找文件, 如果没有命中, 再去磁盘中查找文件。

4.3.1 缓存管理算法

在 Aplaca 云储存系统的缓存模块中, 是使用基于双链表的 LRU-K 算法来对缓存管理, 如图 4 2 所示, LRU-K 算法中的 K 代表最近使用的次数, 因此可以认为 LRU 是“LRU-1”。LRU-K 算法的主要目的是解决 LRU 算法常出现的“缓存污染”问题, LRU-K 算法核心思想是将“最近使用过 1 次”的判断标准扩展为“最近使用过 K 次”。相比 LRU, LRU-K 需要多维护一个访问历史队列, 用于记录所有缓存数据被访问的历史。只有当访问历史队列里的数据访问次数达到 K 次的时候, 才将数据放入缓存。当需要淘汰数据时, LRU-K 会淘汰第 K 次访问时间距当前时间最长的数据。详细实现如下:

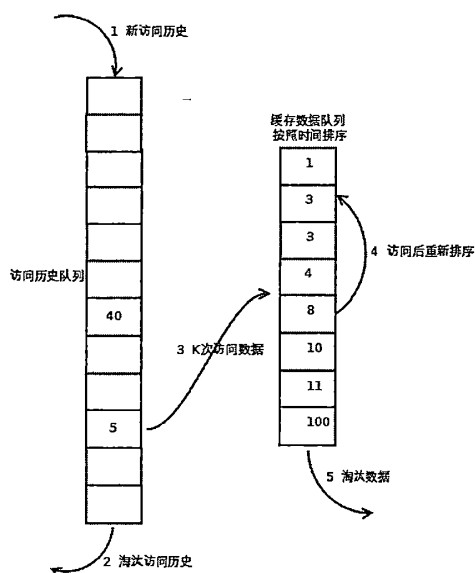


图 4.2 LRU-K 算法

- 1 数据第一次被访问，将被加入到访问历史队列；如果数据索引存在于访问历史队列且访问次数没有达到次数阈值 K ，则该数据的访问次数加一，并把该数据索引移到队尾。
- 2 如果数据在访问历史列表里后没有达到访问次数阈值 K ，则按照 LRU 算法的淘汰规则淘汰，
- 3 当访问历史队列中的数据访问次数达到访问阈值 K 后，将数据索引从访问历史队列删除，并将数据从磁盘缓存到缓存队列中，缓存队列重新按照时间排序；
- 4 缓存数据队列中被再次访问后，重新排序；
- 5 需要淘汰数据时，淘汰缓存队列中排在末尾的数据，即：淘汰“倒数第 K 次访问离现在最久”的数据。

4.3.2 缓存区管理机制

在 client 与 storage server 之间增加缓存层，storage server 在响应 client 文件下载请求之前判断文件是否存在缓存区中。如果是，直接从缓存区读取文件；否则，从磁盘加载文件并判断文件是否满足加入缓存区的要求；

为了缓存区设计的简洁性，缓存区的管理遵循以下几条原则：

(1) 本系统是为满足移动互联网应用的需求而设计，系统默认开启缓存区。storage server 会记录最近 1000 个被下载文件的文件名和下载次数，当某一个文件在最近的 40 分钟之内的下载次数大于 30 次（时间与次数可根据实际情况在配置文件中修改），并且小于缓存区的文件大小阈值以及缓存区有足够空间，则把文件加入缓存区。

(2) 使用基于双链表的 LRU-K 算法进行缓存的管理。

(3) 缓存区更新策略 当缓存区加入的缓存文件大小没有达到预设阈值时，定时从双链表的表尾从后往前遍历，当发现某个文件在最近 50 分钟之内没有被访问时（时间可根据实际情况在配置文件中设定），

释放对应缓存区并将该文件索引从链表中删除。当缓存区加入的缓存大小没有达到预设阈值时，从双链表表尾往前遍历，删除三分之一的缓存索引（删除索引比例可根据实际情况在配置文件中设定），并释放对应缓存区。

（4）记录文件名和下载次数数据结构更新策略 文件被下载，下载次数记录加 1；文件索引从双链表中删除，下载次数记录被重置。

（5）缓存区大小默认是 Storage Server 总内存的 20%（可根据实际情况在配置文件中修改）。

（6）文件大小大于缓存区总大小的文件不会被缓存。

（7）定时写缓存日志，用于缓存区宕机恢复。

4.3.3 缓存区下载文件流程描述

增加缓存后的文件下载流程如图 4 3 所示。具体算法描述如下。

输入：被下载文件的 ID

输出：文件是否在缓存区的布尔值

步骤：

（1）client 向 tracker server 发送文件下载请求。

（2）tracker server 根据动态负载均衡机制返回可用的 storage server 的信息。

（3）client 根据得到的 ip 地址和端口号连接该 storage server 并发送文件下载请求，storage server 解析文件名，得到被下载文件的信息。

（4）根据配置文件的对应配置项判断缓存区是否开启。

（5）根据内存中保存的最近下载文件的次数是否超过预设的阈值来判断近期的文件下载重复率是否较高，如果是，继续下一步，否则返回 false，直接从磁盘读取文件，然后发送给客户端。

（6）storage server 遍历缓存区的索引信息，根据文件 ID 判断缓存区是否命中该文件，如果命中，则从缓存区读取该文件，同时更新缓存区的索引信息，将该文件的索引信息放在缓存索引双链表的最前面位置，执行第八步。否则，执行下一步。

（7）把文件 push 进磁盘读写队列，磁盘读写线程检测到磁盘读写队列有任务到来，开始读取文件内容。与此同时，利用缓存区管理规则，确认是否把文件加入缓存区。

（8）写入日志文件，更新已下载文件数的统计信息，更新下载文件成功数的统计信息。

（9）更新统计近期文件下载次数数的数据结构，将该文件对应的项的下载次数加 1，如果不存在该项，并满足缓存管理规则，则新插入该条目。

（10）关闭连接，文件下载结束。

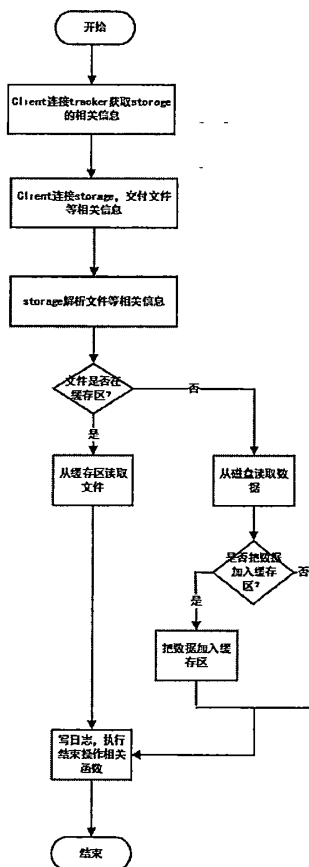


图 4 3 带缓存区的文件下载流程图

4.3.4 缓存区文件更新与删除

文件更新（图 4 3）与文件删除（图 4 4）时，首先判断缓存区是否打开以及文件是否在缓存区，如果没有打开或者文件不在缓存区，则按照 FastDFS 分布式文件系统原有的文件操作逻辑操作文件。如果缓存区已经打开且文件在缓存区，系统对文件进行修改操作时，用修改后的文件替换缓存区中修改前的文件，并更新缓存文件索引,如果修改出错，则需从磁盘中删除被修改的文件，并从缓存区删除该文件；系统对文件进行删除操作时，删除相应的缓存区并删除缓存文件索引。

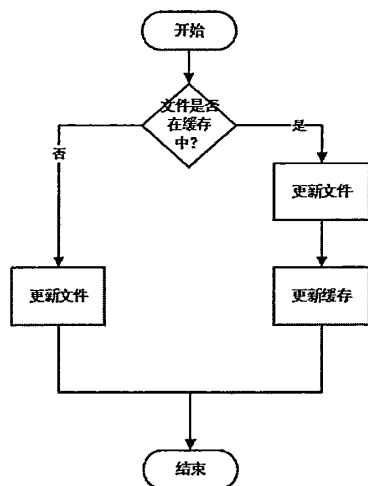


图 4.4 文件更新

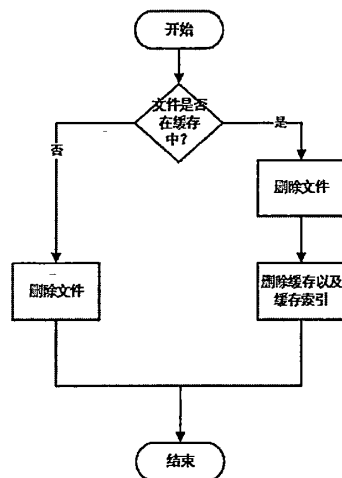


图 4.5 文件删除

4.3.5 缓存区数据结构

缓存区设计主要包括两个部分，一个是管理文件下载次数以及缓存区准入机制的文件下载历史模块，另一个是管理缓存区文件加入与淘汰的缓存索引模块，缓存区数据结构设计主要包括以下几个数据结构：

(1) 文件索引信息

```

struct MEMITEM
{
    char* FileName,
    long  FileSize,
    FILE* FilePointer;
    time_t tm,
    int    fileDownloadNum,
    MEMITEM* front,
    MEMITEM* next,
}, //每个文件索引信息结构体
  
```

(2) 统计文件下载次数

```

struct FileHistoryItem {
    char* fileName;
    int    fileDownloadNum,
    struct fileHistoryItem *front;
    struct fileHistoryItem *next,
};
struct MEMITEM *head=NULL, *rear=NULL; //文件索引信息的头指针和尾指针
static bool isOpenCache;                //系统是否开启缓存模块
static long memCacheSize;               //已经使用的缓存区大小
  
```

4.4 负载均衡管理和优化模块

负载均衡技术是建立在集群上的一种计算机调度技术，它提供了一种透明、高效、廉价的方法，提高了整个

集群的吞吐量、计算能力，增加了整个集群的可用性和可伸缩性。一个具有负载均衡功能的计算机集群的核心之一就是负载均衡算法，负载均衡算法通常分为两类：静态负载均衡算法和动态负载均衡算法。静态负载均衡算法不考虑集群系统的实时运行状态，而只是预先决策计算任务的分配方式。动态负载均衡算法它依靠集群系统当前的负载情况，合理地根据计算机剩余计算能力来分配计算任务。

FastDFS 分布式文件系统只有轮询、选择最大剩余空间、指定服务器等几种简单的静态负载均衡方式，实践表明，相较于静态负载均衡方式，动态负载均衡方式在大多情况下会有更好的性能表现，在 Alpaca 云存储系统中引入动态负载均衡机制，将大大提高存储系统的吞吐量以及可用性。本文所用到的基于动态反馈的负载均衡算法^[9]就是属于动态负载均衡算法的一种。基于动态反馈的负载均衡算法这里主要根据 CPU 使用率、内存使用率、磁盘使用率、网络带宽使用率等因素综合评估以确定响应当前文件存取请求的 storage server。

4.4.1 动态反馈机制

（一）节点负载状况评价

对于节点负载状况评价，目前比较一致的算法是，根据对服务器节点性能影响较大的指标的使用状况来评价节点的负载^[12]。本文亦采用这样的计算方法，并与计算节点性能选取的指标一致，主要从 CPU 使用率 $L(C_i)$ 、内存使用率 $L(M_i)$ 、磁盘使用率 $L(D_i)$ 、网络带宽使用率 $L(N_i)$ 等指标来评价节点 S_i 的负载 $L(S_i)$ 。

$$L(S_i) = r_1 * L(C_i) + r_2 * L(M_i) + r_3 * L(D_i) + r_4 * L(N_i)$$

$$i = 0, 1, \dots, n-1, \quad \sum_{j=1}^4 r_j = 1$$

r 是各项指标的权值参数，反映各个指标对不同类型服务的影响程度，而且权值参数与服务类型有关，不同的服务类型选取的权值参数也不相同。此外，该参数在实际的应用过程中，可以根据系统运行情况进行调节，以期达到更好的负载均衡效果^[12]。

（二）节点处理能力评价

在进行负载均衡计算时，如果服务节点是异构的，则不仅要考虑节点的负载情况，而且也必须要考虑到节点的处理能力，这样能够令处理能力高的节点承担更多的任务^[12]。对于节点的处理能力 $C(S_i)$ ，主要考虑以下指标：CPU 数量 n ，CPU 频率 $C(C_i)$ 、内存容量 $C(M_i)$ 、磁盘容量 $C(D_i)$ 、网络吞吐量 $C(N_i)$ ，storage server 收集到本机的 CPU 数量 n ，CPU 频率 $C(C_i)$ 、内存容量 $C(M_i)$ 、磁盘容量 $C(D_i)$ 、网络吞吐量 $C(N_i)$ 后，借助其心跳协议，每隔一定的时间（默认 30s）把这些数据打包进 storage server 原有的心跳数据包中，然后发送给 tracker server。由于这些指标无法直接表示该节点的处理能力，在 tracker server 端需要采用如下函数来进行转换。

$$C(S_i) = k_1 * n * C(C_i) + k_2 * C(M_i) + k_3 * C(D_i) + k_4 * C(N_i)$$

$$i = 0, 1, \dots, n-1, \quad \sum_{j=1}^4 k_j = 1$$

由于节点的各项指标不同，对各类服务的处理能力也不同。所以，我们在这里引入了参数 k ，用以强调某一类服务对各项指标的依赖程度。若当前参数 k 不能很好地反映节点的负载，系统管理员可以对系数不断地修正，直到找到贴近实际情况的一组系数。

(三) 负载均衡算法

基于上述的服务节点各项参数的计算方法，采用以下算法来分配任务：

(1) 设定一个阈值 f 。

(2) 对于每一个新到来的任务请求，选取节点 S_m 加入候选集合 J 中， S_m 的负载量 $L(S_m)$ 满足条件：

$$L(S_m) = \min\{L(S_i)\}, i = 0, 1, \dots, n-1$$

如果其他任意节点 S_i 负载量满足条件

$$L(S_i) < L(S_m) + f, i = 0, 1, \dots, n-1$$

则将 S_i 加入到集合 J 中。

(3) 计算候选集合 J 中各个节点的分配任务的概率 $P(S_k)$ ，这是根据节点的处理能力 $C(S_k)$ 来计算的：

$$P(S_k) = C(S_k) / \sum_{i=0}^{n-1} C(S_i), k = 0, 1, \dots, n-1, S_i \in J, S_k \in J$$

(4) 根据候选集合 J 各个节点的概率，将任务分配到 J 中的一个节点上。

4.4.2 新负载均衡算法具体应用

新的负载均衡算法在选择响应任务节点时，依然由 Tracker Server 负责 Storage Server 的选择，Tracker Server 首先会进行组内选择 计算每个组内 Storage Server 的节点负载评价值 $L(S_i)$ ，选择负载评价值较小的两个 Storage Server（可根据实际情况调整）加入预评价队列。然后把预评价队列作为新负载均衡算法的输入值，经过新负载均衡算法对节点的评价，得到响应服务请求的输出值，即用来响应服务请求的节点队列。

4.4.3 新负载均衡算法伪代码描述

通过上文的分析，描述了新的负载均衡算法的基本思想和 Storage Server 的评价方法后。下面给出新算法的伪代码。

```

for(each group in所有的组)
{
    计算该group内Storage Server的负载评价值;
    根据负载评价值对Storage Server排序;
    选出负载评价值最小的两个Storage Server (可根据实际情况调整)
    加入预评价队列nodeList;
}
while(遍历nodeList)
{
    计算Storage Server处理能力评价值;
    if (node in nodeList满足加入候选集合J的条件)
    {
        node加入候选集合J;
    }

    根据处理能力评价值计算候选集合队列J中各node的分配任务的概率P，并把P加入pList;
}
对pList中的节点进行排序;
选择pList中概率较大的M个节点响应请求;

```

4.4.4 负载均衡算法数据结构

```

struct LoadCapacity
{
    float CPUUsage,
    float MemoryUsage,
    float DiskUsage,
    float NetworkUsage,
}, //节点负载相关数据

struct ProcessingCapacity
{
    int CPUNum,
    int CPUFrequency,
    int MEMCapacity,
    int DiskCapacity,
    int NetWorkCapacity,
}, //节点处理能力相关数据结构

struct NodeInfo
{
    int group_num,
    int id,
    int storage_port,
    char* ip_addr,
    struct LoadCapacity load_capacity,
    struct ProcessingCapacity processing_capacity,
}, //节点相关详细信息

struct Candidate_Node
{
    struct NodeInfo node,
    float probability,
}, //用于响应请求的候选节点数据结构

struct Candidate_Node Candidate_List[MAX_ITEM], // 用于响应请求的候选队列
int loadFactor[4], //节点负载权重参数
int abilityFactor[4], //节点处理能力权重参数

```

4.5 大文件存储管理和优化模块

Alpaca 云存储系统是基于 FastDFS 设计的一款适用于移动互联网环境的分布式文件系统，FastDFS 特别适合小文件的存储，如果传输的文件较大，应该考虑文件分块技术。首先，使用文件分块技术能降低对移动设备性能以及网络可靠性的要求。其次，如果在某台 storage server 宕机的情况下，只要同组 storage server 同步过来一个文件块，本机就可对外提供该文件的数据服务，提高了整个系统的可用性和吞吐量。另外，使用文件分块技术更易于存储系统应用较前沿的负载均衡算法成果，进一步提升系统可用性与可靠性。另一方面，随着数据量爆炸式增长，推动数据流量大幅增高，人们的数据传输成本也日益增高。然而，FastDFS 并没有在缓解存储系统的空间增长问题，降低数据在网络中的传输成本等方面做考量。

为了解决上述问题，基于 FastDFS 性能优化实现的 Alpaca 云存储系统使用了重复数据删除技术以及文件分块

技术，用于提高整个存储系统的可用性和吞吐量，降低用户的数据成本。

4.5.1 重复数据删除技术实现方式

根据重复数据删除处理发生的位置，可以分为源端重复数据删除（source deduplication）和目标端重复数据删除（target deduplication）两种类型^[11]。源端重复数据删除是指数据在从源端传输到存储设备之前进行重复数据删除，利用源端重复数据删除技术，一方面，可以从源端消除冗余数据，达到对存储空间利用率进行优化的目的。另一方面，可以减少客户端与服务器端之间传输的数据量，并为数据冗余备份节省网络带宽，进而降低能量消耗和网络成本。目标端重复数据删除技术是指在存储系统中进行重复数据删除操作，这种方式可以利用存储系统高性能软硬件的优势，快捷高效地进行重复数据删除，但是用户并不能感受到这种方式带来的方便快捷。

本文基于移动互联网应用场景，遵循降低网络成本和时间成本的原则，选用了基于源端的重复数据删除技术。

4.5.2 Fingerdiff 算法具体实现

本系统的重复数据删除采用 Fingerdiff 算法，并在客户端实现 Fingerdiff 算法。客户端在上传文件之前，先基于文件内容把文件分成若干数据块，然后运用指纹技术对数据块进行相同数据检测及相同数据删除，整个过程主要包含以下 3 个步骤。

- a 首先用 CDC 算法将文件数据切分成若干个子块，
- b 然后用 FingerDiff 算法将所有连续的非重复子块按预先设定的合并子块个数进行合并，形成超级块；
- c 用哈希函数计算每一个超级块的指纹，将其与数据块指纹索引表中的指纹进行比较，如果超级块的指纹已经存在，就将其对应的内容删除，否则将超级块恢复为未合并前的多个小块，计算每一个小块的指纹，查询这些指纹在数据块指纹索引表中的存在性，存储那些指纹不存在于数据块指纹索引表中的小块的内容，并将其指纹也插入到数据块指纹索引表中，对于其余的超级块仍然保持合并状态。

运用 Fingerdiff 算法处理文件后将得到若干数据块，然后打包成一个数据包交由文件分块模块处理。

4.5.3 文件分块实现

从 FastDFS 方面来说，由于 FastDFS 的文件名策略，使得 FastDFS 对小文件的存储性能较好，但是在移动互联网时代，FastDFS 对大文件存储的固有缺陷是一个无法回避的问题。一方面，文件的完整上传所消耗的时间最短，但是上传失败后要重新上传整个文件，而移动互联网环境中，网络不稳定是一种常态，文件的完整上传所带来的弊端将会严重影响用户体验。如果使用了文件分块技术，在上传失败需要重新上传的情况下，只需上传剩下的没有上传成功的文件块，效率将会有所提高，在文件下载的时候也是同样如此；另外，在 FastDFS 集群中，大文件的冗余备份会占用较多的 CPU 时间，而且在大文件同步完成前，短时间除源服务器外的机器无法对外提供该文件的下载、修改等服务。另外，当大文件所在的服务器宕机之后，从组内其他服务器把该大文件同步过来则需要较长的时间，因此该服务器短时间内无法对外提供服务，这将影响整个系统的性能。在这种情况下，文件分块存储同样值得借鉴。如下图 4-6 所示，在大文件存储管理和优化模块中，文件大小如果超过了系统设定的阈值，

则应用大文件分块技术把消重后的数据包文件分成 M 个大小相同的文件块，如果不能等分成 M 块，则分成 $M-1$ 块大小相同的文件块，余下的数据则分配给第 M 块。然后把分块信息上传给 tracker server。

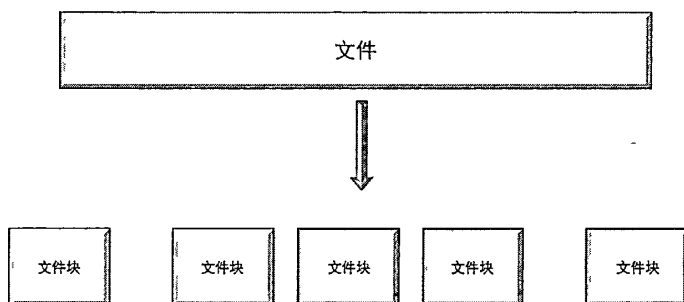


图 4-6 大文件分块

基于以上提到的问题，本文使用重复数据删除技术和大文件分块技术，设计了大文件存储管理和优化模块，在上传文件的时候，先判断文件大小是否超过系统预设阈值，若超过阈值，则把文件在源端进行重复数据删除处理，然后进行分块储存，否则，按 FastDFS 原有的文件上传逻辑上传文件。

下面分别从大文件的上传，文件的删除，文件的修改三个文件操作阐述 client 端、tracker server 端、storage server 端的大文件处理流程。

4.5.4 文件上传

文件上传的流程如图 4-7 所示，tracker server 初始化开辟存储数据块指纹索引表的内存，每次接受到客户端上传文件的请求后，先根据文件的 MD5 值判断存储系统中是否以及存在该文件，如果存在，则建立该文件与存储系统中 MD5 值相同的文件的 ID 映射，然后进行文件上传成功后的相关操作，如果存储系统中不存在该文件，则先判断文件大小是否超过系统预设阈值，若超过阈值（默认 300M），则先对文件进行重复数据删除，此时将得到一个删除了重复数据后的数据包，对该数据包进行是否超过系统阈值判断，如果没有超过阈值，则按原文件上传 API 处理文件；如果超过了系统阈值，则运用大文件分块技术把消重后的数据包文件分成 M 个文件块，tracker server 根据负载均衡方式返回 N 个不同组的 storage server，那么每个 storage 服务器需要上传的文件块数是 M/N ，然后 client 依次与 tracker server 返回的 storage server 建立连接并上传文件。最后，client 把每一个分块的文件 ID、指纹等信息发送给等待的 tracker server。

4.5.5 文件下载

文件下载流程如图 4-8 所示，client 向 tracker server 发送文件下载请求，tracker server 查找文件 ID 列表并根据负载均衡方式返回给 client N 个可用的 storage server。client 开辟 2 个线程（可根据实际需要调整线程数量），每个线程依次向 storage server 发送请求，下载全部文件至本地，然后合并为一个文件。

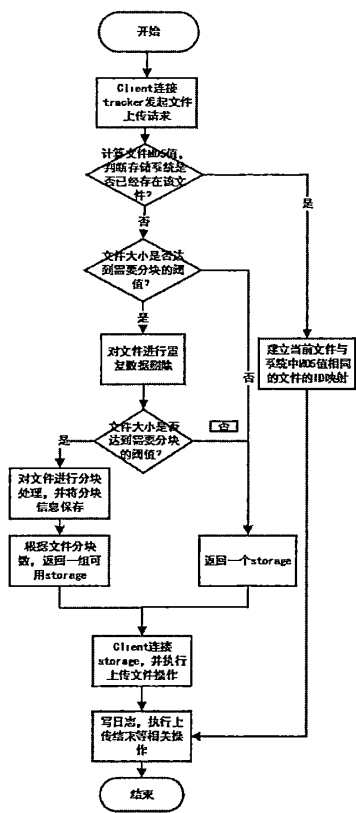


图 4-7 大文件分块上传

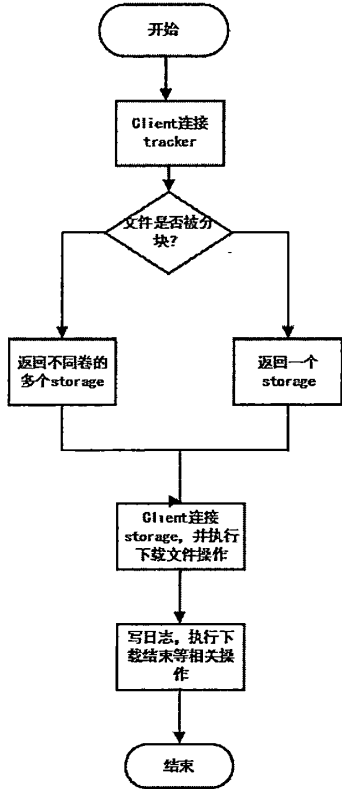


图 4-8 大文件分块下载

4.5.6 文件删除

Client 向 tracker server 发送文件删除请求, tracker server 向 Client 返回文件所在的 storage server 以及子文件列表, 最后 client 依次向 storage server 发出删除请求, storage server 调用原来删除文件的 API 删除文件。

4.5.7 重复数据删除及大文件分块数据结构

a. 重复数据删除数据结构

```
/* deduplication logic block entry */
typedef struct _dedup_logic_block_entry
{
    unsigned long long block_offset,
    unsigned int block_len,
} dedup_logic_block_entry;
```

```
/* deduplication package header */
typedef struct _dedup_package_header
{
    unsigned int block_size,
    unsigned int block_num,
    unsigned int blockid_size,
    unsigned int magic_num,
    unsigned int block_z,
    unsigned int file_num,
    unsigned long long ldata_offset,
    unsigned long long metadata_offset;
} dedup_package_header;

/*deduplication metadata block entry*/
typedef struct _dedup_metadata_block_entry
{
    unsigned int superblock_id,
    unsigned int superblock_offset;
    unsigned int superblock_len,
} dedup_metadata_block_entry;

/* deduplication metadata entry header */
typedef struct _dedup_entry_header
{
    unsigned int path_len,
    unsigned int block_num,
    unsigned int entry_size,
    unsigned int last_block_size,
    unsigned long long old_size,
    int mode;
    time_t atime;
    time_t mtime;
} dedup_entry_header;

/* deduplication operations */
enum DEDUP_OPERATIONS
{
    DEDUP_CREAT = 0,
    DEDUP_EXTRACT,
    DEDUP_APPEND,
    DEDUP_REMOVE,
    DEDUP_LIST,
    DEDUP_STAT
},
```

b. 大文件分块数据结构

```

typedef struct
{
    char FirstFileID[FILEIDSIZE];    //第一个分块ID
    char* FileID[BLOCKNUM];          //所有的分块ID
} BLOCKINFO,                        //tracker server端的分块信息记录

struct blockBinLogRecord
{
    char op_type, //操作类型
    char FirstFileID[FILEIDSIZE];
    char* FileID[BLOCKNUM];
    time_t timestamp,
    long src_filename_len,
    int sub_fileNumber,
}, //文件块日志信息

struct block_tag
{
    char src_Filename[128], //被分割的文件名
    char sub_Filename[128], //子文件名
    long nFileSize, //文件长度
    int fileNumber, //总分割数
    int fNo, //子文件序号
    long fStart, //分割的起始位置
    long fEnd, //分割的起始位置
},

```

4.6 本章小结

本章分别详细介绍了 FastDFS 在缓存、负载均衡以及大文件存储管理方面的不足，基于此提出了对应的解决方案。然后阐述了 Alpaca 云存储系统的整体架构，并基于此架构设计了三大模块。一是针对互联网环境中热文件访问的性能问题，制定了一系列的缓存区管理规则，并将其融入缓存机制管理和优化模块中；二是针对 FastDFS 原有负载均衡方式无法胜任 Alpaca 云存储系统全球化应用场景的问题，设计并实现了负载均衡管理和优化模块。三是针对 FastDFS 对大文件支持不足的问题，设计并实现了大文件存储管理和优化模块。

第五章 Alpaca 云存储系统模块测试

基于 FastDFS 性能优化设计的 Alpaca 云存储系统是一款为移动互联网用户提供云存储服务的产品，为了验证系统的有效性和可靠性，本章将按照移动互联网产品的性能评价指标，对系统进行单模块逐一测试以及多模块集成测试，并分析测试结果，给出性能评价。

5.1 测试环境

5.1.1 测试环境架构

为了评价基于 FastDFS 的 Alpaca 云存储系统的性能，本文在 VMware 虚拟环境下搭建了 6 台虚拟机，如图 5.1 所示，一台作为 client，一台作为 tracker server，另外四台作为 storage server（四台 storage server 分为三组，组一两台，组二和组三分别一台），然后分别部署上 centos 6.5 以及改进的 FastDFS 环境。

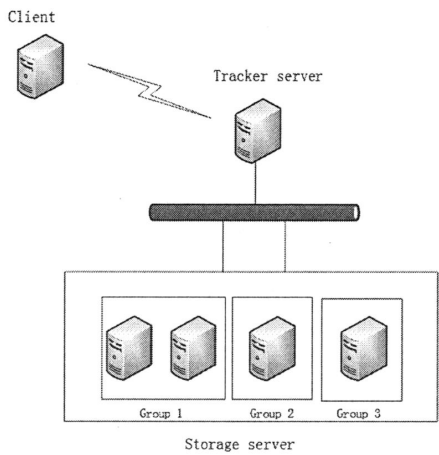


图 5.1 FastDFS 测试架构图

搭载虚拟机的物理机配置如下：

操作系统：Windows 7 64 位

处理器：Intel(R) Core(TM) i5-2410M CPU @ 2.30GHz

内存：10G

硬盘：500G

各虚拟机配置如下表所示：

表 5.1 测试环境

角色	配置
Client	CentOS6.5 256M 内存 10G 硬盘 1 个单核 CPU 100M 带宽
Tracker server	CentOS6.5 1G 内存 10G 硬盘 1 个双核 CPU 100M 带宽
Storage server (group 1)	CentOS6.5 128M 内存 10G 硬盘 1 个单核 CPU 50M 带宽

Storage server (group 1)	CentOS6 5 256M 内存 20G 硬盘 2 个单核 CPU 100M 带宽
Storage server (group 2)	CentOS6 5 512M 内存 10G 硬盘 2 个双核 CPU 50M 带宽
Storage server (group 3)	CentOS6 5 1G 内存 20G 硬盘 1 个双核 CPU 100M 带宽

5.1.2 改进的 FastDFS 环境部署

(1) libfastcommon

libfastcommom 是安装 FastDFS 之前必须安装到 Linux 环境中的库文件集合，它从开源项目 FastDFS 中提取出来的公共库，它的主要功能是为 FastDFS 提供字符串处理、日志、base64 的编码与解码、对象池等功能，是 FastDFS 稳定运行的保障。

(2) Libevent

Libevent 是一个用 C 语言编写的、轻量级的开源高性能网络库，适用于 windows、linux、bsd 等多种平台，内部使用 select、epoll、kqueue、IOCP 等系统调用管理事件机制。主要有以下几个亮点：事件驱动（event-driven），高性能，轻量级，专注于网络，不如 ACE 那么臃肿庞大；源代码相当精炼、易读；跨平台，支持多种 I/O 多路复用技术，epoll、poll、dev/poll、select 和 kqueue 等，支持 I/O，定时器和信号等事件，注册事件优先级。

Libevent 编译库会判断 OS 支持哪种类型的事件机制（select、epoll 或 kqueue），然后条件编译相应代码，供上层使用的接口仍然是保持统一的。libevent 支持用户使用三种类型的事件，分别是网络 IO、定时器、信号三种，在定时器的实现上使用了最小堆的数据结构，以达到高效查找、排序、删除定时器的目的，IO 和信号的实现均使用了双向队列。Libevent 提供了 DNS，HTTP Server，RPC 等组件，HTTP Server 可以说是 Libevent 的经典应用。

(3) FastDFS 安装过程

a.libfastcommon 安装

```
cd /usr/local/libfastcommon-master
/make sh
/make sh install
#libfastcommon so 默认安装到了/usr/lib64/libfastcommon so, 而 FastDFS 主程序设置的 lib 目录是/usr/local/lib,
所以设置软连接。
ln -s /usr/lib64/libfastcommon so /usr/local/lib/libfastcommon so
ln -s /usr/lib64/libfastcommon so /usr/lib/libfastcommon so
ln -s /usr/lib64/libfdfsclient so /usr/local/lib/libfdfsclient so
ln -s /usr/lib64/libfdfsclient so /usr/lib/libfdfsclient so
```

b. libevent 安装

```
tar zxvf libevent-2.0.18-stable.tar.gz
cd libevent-2.0.18-stable
./configure
make && make install
#为 libevent 创建软链接到 /lib 库下，64 位系统对应 /lib64
ln -s /usr/local/lib/libevent* /lib/
ln -s /usr/local/lib/libevent* /lib64/
```

c. FastDFS 安装

```
tar zxvf FastDFS_v3.06.tar.gz
cd FastDFS
./make.sh
./make.sh install
```

(4) FastDFS 基本配置

a. 配置及启动 Tracker Server

```
mkdir /home/fastdfs
vi /etc/fdfs/tracker.conf
修改下面指定的参数为指定值：
disabled=false      #启用配置文件
port=22122          #设置 tracker 的端口号
base_path=/fdfs/tracker #设置 tracker 的数据文件和日志目录（需预先创建）
http_server_port=8080 #设置 http 端口号
#如需要进行性能调优，可以参照附录的配置文件的详细说明。
Tracker Server 的启动：
cd /usr/local/bin
fdfs_trackerd /etc/fdfs/tracker.conf
```

b. 配置及启动 Storage Server

```
cd /etc/fdfs
vi storage.conf
一般只需改动以下几个参数即可
disabled=false      #启用配置文件
group_name=group1   #组名，根据实际情况修改
```

```
port=23000          #设置 storage 的端口号
base_path=/fdfs/storage  #设置 storage 的日志目录（需预先创建）
store_path_count=1    #存储路径个数，需要和 store_path 个数匹配
store_path0=/fdfs/storage  #存储路径
tracker_server=$(tracker server IP) 22122 #tracker 服务器的 IP 地址和端口号
http server_port=8080    #设置 http 端口号
如需要进行性能调优，可以参照附录的配置文件的详细说明。
Storage Server 的启动：
cd /usr/local/bin
fdfs_storaged /etc/fdfs/storage conf
```

c. 配置 client

```
vi/etc/fdfs/client conf
修改下面指定的参数为指定值：
base_path=/home/fastdfs
tracker server=$(tracker server IP): 22122
http trackerserver_port=8080
```

5.2 测试方案

基于 VMware 平台和 centos 操作系统搭建 6 台虚拟机，用 VMware 平台通过客户端的多线程模拟真实环境中的高并发请求，对基于 FastDFS 的 Alpaca 云存储系统从两方面进行测试，一是，利用 6 台虚拟机分别对单个模块进行测试。然后再同时开启三个模块，对系统进行综合测试。最后根据缓存命中率、I/O speed、服务器响应时间、文件下载速度、文件下载时间等指标分别对模块表现以及系统整体表现进行分析。

5.2.1 模块性能测试

5.2.1.1 缓存功能性能测试

本文为了测试缓存模块的有效性，利用客户端不间断地对存储系统中的 5G word 文件、TXT 文本文件、Windows 安装程序、图片等常规文件发起下载请求，分别用 30-5000 请求数去模拟真实情况下的文件下载过程，通过测试所得数据，求得缓存命中率。

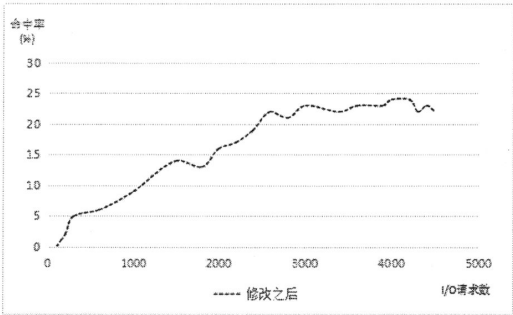


图 5.2 缓存命中率

实验结果分析

通过对五组实验数据的分析发现，缓存模块的命中率约为 24%，这主要是因为并不是所有数据都缓存，而是只是缓存热数据。当缓存区大小改变以及文件粒度变化时，缓存模块的缓存命中率也会得到相应的改变，但是缓存命中率曲线的变化趋势是相同的。从图可知，刚开始缓存的命中率都比较低，这是因为设计的缓存策略并非是对每个文件都进行缓存，而只在某个文件达到一定热度时才进行缓存。另外，当请求数较高时，命中率会有小范围下降，这是由于热文件增加，缓存替换加快引起的。

5.2.1.2 负载均衡功能性能测试

为了验证负载均衡模块的有效性，本文测试了修改后的动态负载均衡算法以及 FastDFS 轮询负载均衡算法的平均应答时间，以验证优化后的负载均衡算法达到了预期效果。客户端分别开辟 1、10、30、60、100、200、300 个线程，分别对修改后分布式文件系统发起下载请求，下载的文件大小为 50M，对平均应答时间进行测试十次，得出结果如图 5.3 所示。

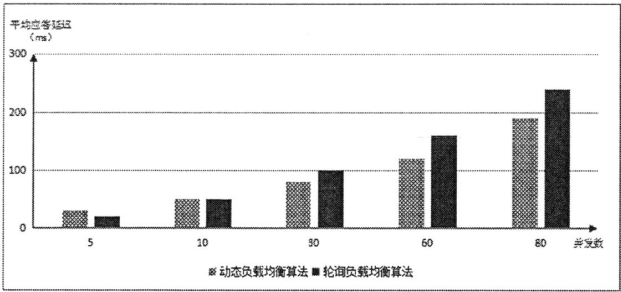


图 5.3 平均应答延迟

实验结果分析

如图所示，与轮询负载均衡算法相比，新的动态负载均衡算法确实能够有效地减少平均应答时间，当请求数较小时，这些改善不太明显，甚至效果比轮询负载均衡算法要差。这主要是因为新的动态负载均衡算法复杂度较高而带来的负面影响，随着请求数逐渐增加，动态负载均衡算法的优势才开始显现出来。

5.2.1.3 重复数据删除功能模拟测试

为了考察大文件存储管理和优化模块中重复数据删除功能的消数据重率以及时间性能，本文对 fingerdiff 算法进行了仿真实验，实验时选取了 Alpaca 云储存系统将要存储的且比较常见的设备色彩特性文件、影像文件以及 word 文件作为样本数据进行测试，所有的样本数据都是模拟日常环境中的数据。并把这些文件打包成一个文件进行测试，样本数据共有 7 组，单文件大小逐渐由 39KB 增长到 50MB，合并文件大小从约 200KB 增长到约 120MB。表 5 2 所示是测试样本列表。

表 5 2 测试样本列表

样本 编号	文件内容	样本大小
1	39KB 的设备色彩特性文件、51KB 的影像文件、110KB 的 word 文件	200KB
2	45KB 的设备色彩特性文件、212KB 的影像文件、121KB 的 word 文件	378KB
3	57KB 的设备色彩特性文件、934KB 的影像文件、613KB 的 word 文件	1604KB
4	1 1MB 的设备色彩特性文件、2 3MB 的影像文件、1 2MB 的 word 文件	4 6MB
5	5 3MB 的设备色彩特性文件、10 7MB 的影像文件、4 6MB 的 word 文件	20 6MB
6	12MB 的设备色彩特性文件、19MB 的影像文件、6 3MB 的 word 文件	37 3MB
7	39MB 的设备色彩特性文件、51 1MB 的影像文件、10 2MB 的 word 文件	100 3MB

实验结果分析

本文使用了消重率作为评价数据消重算法性能的指标，其计算方法是文件的原始大小除以消重后文件大小 $\text{original_data_size}/\text{eliminated_data_size}$ 。另外，本文还定义消重性能作为评价数据分块算法性能的指标，其计算方法是 $\text{eliminated_data_size}/\text{time}$ ，单位是 KB/S。它表示系统每秒钟消除掉多少 KB 的重复数据。这个评价指标同时考虑了数据分块算法的消重率和消重时间，能够较好地评估云存储应用中数据消重效果。使用上文提到的测试环境，得到性能指标数据如表 5 3 所示。

表 5 3 消重性能（单位 KB/S）

样本大小	消重时间	消重率	消重性能
200KB	0 03	1 0154	101 10
378KB	0 03	1 0046	57 69
1604KB	0 10	1 0857	1266 12

4 6MB	0 19	1 0852	1946 40
20 6MB	0 89	1 2461	4680 97
37 3MB	1 18	1 2911	7298 08
100 3MB	3 87	1 3156	6366 53

5.2.2 系统性能测试

此次测试将优化后的三个模块集成到系统中，然后同时开启三个模块，对系统进行综合测试。首先生成总大小为 5 8G 大小的测试文件集合，然后上传至分布式文件系统，文件的大小和数量如表 5 4 所示。

表 5 4 测试文件列表

文件大小	数量（个）
10KB	100000
100KB	10000
1M	1000
10M	100
50M	20
100M	10

然后对已上传至分布式文件的文件做下载测试，客户端分别开辟 1、10、30、60、100、200、300 个线程，分别对该分布式文件系统发起下载请求，对 I/O speed 和平均应答时间进行测试三次。

5.2.2.1 I/O Speed

在实践中发现，存储系统的性能受并发请求数量的影响较大，当并发量增大时，I/O 速度是逐渐下降的，因此，本文采用多线程模拟真实环境中的高并发环境，系统 I/O Speed 值如图 5 4 所示。

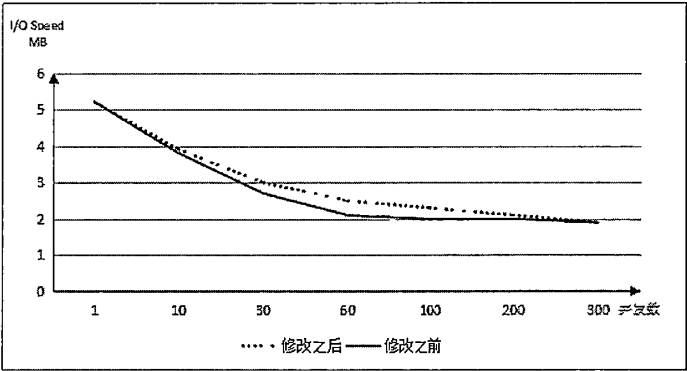


图 5 4 I/O speed

我们从图可知，当请求数较小时，Alpaca 云存储系统相较于 FastDFS 来说，性能的提升并不太明显，具体表现为请求数为 1-20 时。当请求数大于 20 时，改良后的 Alpaca 云存储系统的优势才逐渐表现出来，

当请求数为 20-60 时，由于动态负载均衡算法的存在，存储系统性能得到明显提升。当并发数处于 60 时，缓存的作用开始得到显现，热文件的读取不再是通过磁盘 I/O，而是从内存直接读取，因而并发数处于 60-300 时，性能的提升完全来自缓存层的支持。

5.2.2.2 平均应答响应时间

通过实践可知，平均应答时间可以在一定程度上反应出系统的性能。在系统负载均衡的优化中，通过对比优化前后平均应答时间可以平均负载均衡算法性能。

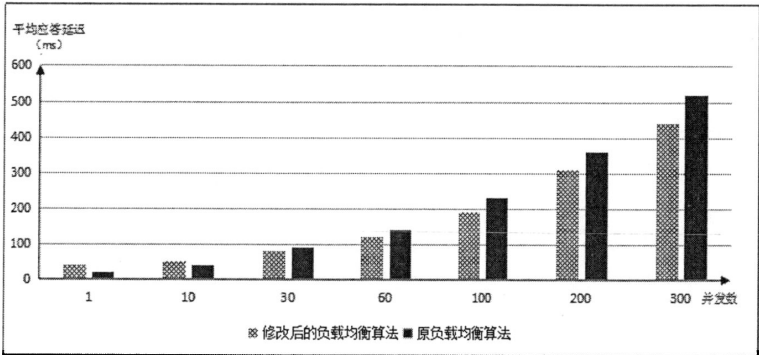


图 5.5 平均应答响应时间

从上图可以看出，当并发数较小时，动态负载均衡算法对系统性能带来的增益并不明显，甚至性能低于原有的负载均衡算法，这主要由于动态负载均衡复杂的算法处理影响了性能。随着并发数的逐渐提高，动态负载均衡算法的优势才显现出来，对服务器节点的跟踪和分析才更精确，对负载分配更均衡。

5.2.2.3 大文件下载时间及下载速率

客户端分别往客户端上传了没有经过压缩处理的 Solr 安装包、360 程序文件夹、视频，然后对大文件进行下载实验。经过测试，大文件的原始数据大小及消重后的数据大小如表 5.5 所示。

表 5.5 大文件消重数据

文件	大小（去重前）	大小（去重后）	消重率
Solr（java 版）文件夹	445M	258M	42%
360 程序安装文件夹	731M	654M	10.5%
视频	995M	960M	4%

经过五次大文件下载实验取平均值，得到大文件下载时间如表所示。

表 5.6 大文件下载时间

文件大小	FastDFS 下载时间(s)	Alpaca 下载时间(s)
445M	85	27
731M	152	73
995M	236	120

从上述可以得出, Alpaca 下载大文件的时间比 FastDFS 下载文件的时间的一半还少, 这主要是由两方面原因构成, 第一, 在源端使用了基于 fingerDiff 算法对数据进行消重, 分别消除了 42%、10.5%以及 4%的重复数据。第二, Alpaca 云存储系统对大文件进行了分块处理, 在下载时使用双线程下载文件。基于以上两点原因, 大大降低了文件的下载时间。

5.3 本章小结

本章首先介绍了基于 VMware 平台和 centos 操作系统搭建的 Alpaca 测试环境相关细节, 拟定用 VMware 平台中客户端的多线程请求模拟真实环境中高并发请求的测试方案, 然后针对基于 FastDFS 优化的三个模块, 从 I/O speed、平均响应时间以及大文件下载时间及下载速率等方面分别对系统进行了单模块测试和多模块集成测试。测试结果表明, 优化后的三个模块集成到系统中后, 相较于 FastDFS, Alpaca 云存储系统的性能有明显提升, 具有较高的实用价值。

第六章 总结与展望

由上文测试数据可知, 基于 FastDFS 性能优化设计的 Alpaca 云存储系统在文件存储方面相较于 FastDFS 分布式文件系统性能有明显提升, 这主要是因为缓存机制管理和优化模块、负载均衡管理和优化模块以及大文件存储管理和优化模块在整个云存储系统中相辅相成, 使其适应于当前复杂多变的网络环境, 但是这三大模块仍存在许多性能问题亟待解决。

6.1 总结

本文以移动互联网环境中的云存储需求为背景, 并认真分析了柯达文件存储业务全球化的需求以及 Alpaca 云存储系统的现实需求, 设计了基于 FastDFS 性能优化的 Alpaca 云存储系统, 本系统的完成摒弃了 FastDFS 部分缺陷, 提升了 FastDFS 在热数据处理、服务器负载均衡以及大文件存储等方面的性能, 使基于 FastDFS 性能优化的 Alpaca 云存储系统能够更好服务于全球移动端和 PC 端的柯达客户。本论文主要工作总结如下:

1、通过发现 FastDFS 热文件存取的不足, 研究和制定包括缓存区开辟前置条件、文件查找流程、缓存区文件淘汰机制、缓存区更新策略等一系列的缓存区管理规则, 并利用这些缓存区管理规则在 client 与 storage server 之间增加缓存机制管理和优化模块, 实现热文件的存取优化。

2、通过发现 FastDFS 负载均衡策略的不足, 结合 CPU、内存、磁盘、网络带宽等资源使用率等因子, 应用现有的负载均衡算法研究成果, 在 tracker server 与 storage server 之间增加负载均衡管理和优化模块, 实现动态反馈调整集群的流量分配和全局负载均衡。

3、通过发现 FastDFS 文件上传机制的不足, 应用现有的文件分块以及数据去重算法研究成果, 形成新的文件上传机制, 在 client 与 tracker server 之间增加大文件存储管理和优化模块, 提高大文件存储性能。

4、通过以上三个方面的研究, 针对 FastDFS 相应设计和实现了缓存机制管理和优化模块、负载均衡管理和优化模块以及大文件存储管理和优化模块, 并统一集成至 Alpaca 云存储系统, 最后通过多次严谨的测试证明, 基于 FastDFS 性能优化设计的 Alpaca 云存储系统达到了高效处理热数据、均衡服务器集群负载请求以及提高大文件存储效率的设计目标, 具有实际的生产环境应用价值。

6.2 展望

本文对分布式文件系统以及云存储技术做了一些研究, 并在这些研究的基础上实现了基于 FastDFS 性能优化设计的 Alpaca 云存储系统, 最后顺利完成了项目启动时所安排的开发任务, 但是由于是第一次参与如此大型的商业项目, 心态以及科研能力的不足, 导致项目的完成并不如当初期望的尽善尽美, 这需要我在今后的工作中进一步深入的研究和完善, 主要包括以下两方面:

首先, 在实际生产环境中, 机器的剩余处理能力总是处于动态变化之中, 因而机器闲置内存大小也是处于动态变化之中, 内存缓存设置过大或者过小都会在一定程度上影响机器的整体性能, 因而根据机器的剩余处理能力动态调整内存缓存大小是非常必要的, 这将在一定程度提升集群的整体性能。

其次, 由于 Alpaca 云存储系统中的客户端大多都是性能较低的移动设备, 因而无法在客户端实

现复杂且性能较好的重复数据删除技术以及数据压缩技术。通常，服务器端的性能是远优于客户端，如果在服务器端实现性能较好的重复数据删除技术以及数据压缩技术，这将减少文件在云端和客户端之间传输时所消耗的带宽，提升用户体验。另外，当前重复数据删除算法对文件类型敏感，而且消重性能也不高，未来的优化中，希望能减少重复数据删除过程的等待时间及增加消重率。

参考文献

- [1] 余庆 分布式文件系统 FastDFS 架构剖析[J] 程序员, 2010(11): 63-65
- [2] 黄晓云 基于 HDFS 的云存储服务系统研究[D] 大连. 大连海事大学, 2010
- [3] HDFS Architecture Guide[EB/OL]. <http://hadoop.apache.org>
- [4] Lustre Home Page[EB/OL] <http://lustre.org/>
- [5] MongoDB Manual[EB/OL] <https://docs.mongodb.com/manual/core/gridfs/>
- [6] Github Mogilefs[EB/OL] <https://github.com/mogilefs>
- [7] Taobao File System Home Page[EB/OL] <http://tfs.taobao.org/>
- [8] 探析网络文件系统的演化与发展[EB/OL]. <http://tech.hexun.com/2011-01-28/127070028.html>
- [9] OpenAFS Home Page[EB/OL] <http://www.openafs.org/>
- [10] Schaller R R Moore's law past, present and future[J] IEEE Spectrum, 1997, 34(6): 52-59
- [11] L D Bois and R Amatruda Backup and Recovery Accelerating Efficiency and Driving Down IT Costs Using Data Deduplication[J] EMC Corporation, Feb. 2010
- [12] 刘健, 徐磊, 张维明. 基于动态反馈的负载均衡算法[J]. 计算机工程与科学. 2003, 25(5): 65-68.
- [13] 朱平, 朱建涛, 高剑刚等 高性能计算存储关键技术研究[J]. Journal of Computer Research and Development, 2011, 48(1) 354-364
- [14] Srinivasan Rashmi KrishnaIyengar, Suma V , Nedu Vaidehi An enhanced load balancing technique for efficient load distribution in cloud-based it industries[C] Chennai, India: Springer Verlag, 2013, 479-485
- [15] He Ruiying, Zhao Yongli, Wang Wei etc Research on the load balancing algorithm based on the datacenters service[C] Shenyang, Liaoning, China Trans Tech Publications, 2012, 420-425
- [16] Chen Wei, Zhang Yu-Fang, Xiong Zhong-Yang. Research and realization of the load balancing algorithm for heterogeneous cluster with dynamic feedback[J] Chongqing Daxue Xuebao, Journal of Chongqing University. 2010, 33(2)· 73-78
- [17] Ning Guo-Qin, Zhu Guang-Xi, Peng Lie-Xin etc Research on hybrid dynamic load balancing algorithm in heterogeneous hierarchical wireless networks[J] Tongxin Xuebao, Journal on Communications 2007, 28(1): 75-81.
- [18] Sivakumar Raghupathy, Sinha Prasun, Bharghavan Vaduvur. Cedar: a core-extraction distributed ad hoc routing algorithm[J] IEEE Journal on Selected Areas in Communications 1999, 17(8)· 1454-1465
- [19] 王亮. 基于 Linux 的分布式文件系统的设计与实现[D] 武汉· 华中科技大学, 2013
- [20] 牛升 分布式文件系统的负载均衡策略研究[D]. 成都: 电子科技大学, 2014.

- [21] 王敬轩 分布式文件系统存储效率优化研究[D]. 武汉·华中科技大学, 2013.
- [22] 叶为民 分布式文件系统自适应负载调度研究[D]. 武汉·华中科技大学, 2013
- [23] 张云升. 面向数据存储的分布式文件系统的研究与设计[D]. 武汉: 武汉理工大学, 2012
- [24] 赵黎斌. 面向云存储的分布式文件系统关键技术研究[D] 西安: 西安电子科技大学, 2011.
- [25] 张大洋 基于云存储的分布式文件系统研究与优化[D] 西安·西安电子科技大学, 2013
- [26] 肖中正, 陈宁江, 魏峻等 一种面向海量存储系统的高效元数据集群管理方案[J] 计算机研究与发展, 2015, (4) 929-942
- [27] 吴静, 戴志涛. FastDFS 针对数据采集应用的改进[J] 软件, 2014, (12)· 123-128
- [28] 张桂刚, 李超, 张勇, 邢春晓 一种基于海量信息处理的云存储模型研究[J] 计算机研究与发展, 2012, 49(z1) 31-36.
- [29] 周江, 王伟平, 孟丹等. 面向大数据分析的分布式文件系统关键技术[J] 计算机研究与发展, 2014, 51(2)· 382-394.
- [30] 郭威 分布式文件系统 ZD-DFS 的设计与实现[D] 杭州·浙江大学, 2006
- [31] George C, Jean D, Tim K D Distributed systems concepts and design(3rd Edition)[M]. Addison Wesley Publishers Limited 2000.
- [32] Russel S, David G, Steve K, et al. Design and implementation of the Sun network file system[J] Artech House, 1988
- [33] Hadoop community Hadoop distributed file system[EB/OL] <http://hadoop.apache.org/hdfs>, 2010.
- [34] FastDFS community. FastDFS[EB/OL]. <http://bbs.chinaunix.net/forum-240-1.html>. 20
- [35] 敖丽, 舒继武, 李明强 重复数据删除技术[J]. 软件学报, 2010, 21(5) 916-929
- [36] 杨春贵, 吴产乐, 彭鸿雁 一种有效的 Web 代理缓存替换算法[J] 计算机工程, 2007, 33(3)· 43-44
- [37] 分布式文件系统客户端关键技术研究[D] 武汉·华中科技大学, 2013
- [38] Ceph 分布式文件系统的研究及性能测试[D]. 西安 西安电子科技大学, 2014
- [39] 一种分布式文件系统在云存储中的研究与实现[D]. 南京·东南大学, 2014.
- [40] 刘立坤等. CorsairFS 一种面向校园网的分布式文件系统[J]. 西安交通大学学报, 2009, 48(3): 43-48
- [41] Object—Based Storage Devices(OSD)workgroup[EB/OL]. <http://www.snia.org/osd>, 2004
- [42] peter J. Braam. The Lustre Storage Architecture[J] White Paper, 2003.
- [43] Panasas Inc. Object Storage Architecture[J] White Paper, 2004
- [44] Intel OSD home page[EB/OL] <http://www.intel.com/labs/storage/osd/tech.html>, 2003
- [45] IBM OSD home page[EB/OL] <http://www.haifa.ibm.com/projects/storage/objectstore/index.html>, 2003
- [46] 孟晓炬, 李一鸣, 卜庆忠. 一种面向存储服务的缓存管理模型[J] 计算机工程, 2009, 35(I)· 30—32.

- [47] R P Wooster, M Abrams Proxy caching that estimates page load delays[J] In Proceedings of ACM SGCOMM1996, 1 996, PP 293-304
- [48] 夏卓群, 项洁, 范开钦, 邓广慧 基于集群的流媒体缓存代理服务器体系结构[J] 计算机应用 2007, 27(11)· 2844-2846

致谢

值此论文完成之际，我要向所有关心、支持和包容我的老师、同学和亲人表示最衷心的感谢！

首先，我要向我的校内导师罗军舟教授以及论张竞慧老师表示深深的感谢，感谢他们在百忙之中抽出时间指导我的课题研究以及论文撰写工作，他们的悉心指导帮助我明确了课题研究方向、理顺了论文写作思路，使我在掌握了许多实用的科研技巧以及研究思路。

另外，我特别感谢我的校外导师朱兴明高级工程师，在柯达实习的这几个月中，朱兴明高工对我的帮助非常大，无论在工作生活中还是在论文的写作过程中，他严谨的科学态度、精益求精的工作作风以及他的博学和无私都深深地感染和激励着我。从课题的选择到项目的最终完成，朱兴明高工都始终给予我细心的指导和不懈的支持。另外，我还要感谢研究生院里每一位服务于学生的老师和同学，是他们的认真负责，让我们每一步都走的如此顺利。

然后，感谢我的母校东南大学，是东大给我提供了发展的平台，她让我发光发亮，绚丽多彩。学校不仅给我们提供了一个安全舒适的学习环境，而且教导我们如何为人处世，独立生活，为我们步入社会提供宝贵的经验，让我从一个懵懂青年蜕变成对祖国建设有用之人。

书到用时方恨少，在这篇论文的写作过程中，我深感自己的水平还非常的欠缺。生命不息，学习不止，人生就是一个不断学习和完善的过程，敢问路在何方？路在脚下！今后，我将继续运用老师们、同事们和同学们教给我的知识，不断超越一个又一个不同时期的自己。

最后，我要感谢我的父母和亲友，有他们的陪伴，才有了我的今天。

朱帅

2017 年 05 月于东南大学

实习情况介绍

在整个读研期间，我总共在两家公司实习过，第一家实习的公司是柯达（中国）投资有限公司，第二家实习的公司是阿里巴巴（中国）投资有限公司。通过在这两家公司参与实际项目的研发，培养了自己善于观察、勤于思考的良好学习习惯以及严谨的科学态度和实际动手能力，使理论与实践得到很好的结合。实习结束后，我认为自己获得的不只是工作技能上的提升，更是促进了自己心理的成熟以及培养了自己对未来的信心。另外，切身接触社会才懂得生活的艰辛，更懂得同学情谊、师生情谊的重要性。

我实习的第一家公司是柯达（中国）投资有限公司，柯达目前致力于利用百年积淀的材料科学以及影像科学成果，研发绿色印染技术以及印染集成解决方案。我在柯达产品研发中心实习了十个月，主要参与了两个项目，第一个是 ColorFlow 项目，ColorFlow 软件是一种智能化色彩解决方案，用于集成典型印刷生产环境中的多种元素和设备，并管理其间复杂的色彩关系。它能够完全存储和自动化精确颜色调整、对齐曲线和轮廓，减少手动任务和操作。第二个是 Alpaca 云存储系统项目，Alpaca 云储存系统主要目的是用于整合全球柯达客户的印刷解决方案，实现对海量印刷过程中所涉的设备色彩特性文件，商业印刷中的包装、封面、LOGO 以及版式等设计稿件，个性化影像业务和景点影像解决方案形成的影像文档等各类文件的高效存储管理。

我实习的第二家公司是阿里巴巴（中国）投资有限公司，我在阿里巴巴国际化事业部的一达通部门实习了三个月，我主要参与的是一达通关务系统以及风险控制系统的研发，一达通部门通过关务、风控等系统提供线上化操作及建立有效的信用数据系统，为中小企业提供专业、低成本的通关、外汇、退税及配套的物流和金融等外贸综合服务。

通过两段实习经历，才真正体会到终身学习的含义，我们迈出校门后，最艰难的学习过程才真正开始，不论在人生的哪个阶段，学习的脚步都不能停歇，要把工作视为学习的殿堂。在这几个月的学习与工作中，我在领导与同事们的帮助和支持下取得了较大的进步，但我深知自己还存在较多的缺点和不足，在工作中经常不能对问题考虑周全，工作灵活度不够，团队意识有待提升，这都有待于在今后的工作中继续改进。

在柯达和阿里巴巴，我迈出了从学生到职场人生的第一步，也是迈向社会的重要一步。我学到了很多书本上学不到的东西，也有了从浅知到熟知，从工作磕磕碰碰到独当一面的转变。因而，我首先要特别感谢我的老师们以及同学们的帮助，让我在面对工作中困难的时候能积极应对、勇敢向前。此外，也要感谢柯达和阿里给我提供良好的实习环境，使我在专业技能上有了较大提升。最后，还有感谢我的项目团队成员，正是因为有大家的相互帮助和学习，共同面对挑战，共同解决问题，我的实习过程才能顺利完成，我也更加认识到了团队精神的可贵，同时也收获了最真挚的友谊。